

Universidade de Vigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D. Nome Alumna/o

para a obtención do Título de Graduado en Enxeñaría Informática

Título do Traballo de Fin de Grado



Xullo, 20XX

Traballo de Fin de Grao N°:

Titor/a:

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática

Agradecimientos

[Texto]

Dedicatoria

[Texto]

Índice

1. Introducción	6
2. Objetivos	7
2.1. Objetivo principal	7
2.2. Objetivos específicos	7
2.3. Alcance y limitaciones	7
3. Antecedentes y contexto	8
3.1. Deep Reinforcement Learning en Ajedrez	8
3.2. Transformers en ajedrez	8
4. Resumen de la solución propuesta	10
5. Marco teórico y práctico. Herramientas empleadas	11
5.1. Marco teórico	11
5.1.1. Diseño Experimental e Hipótesis	11
5.1.2. Entorno: Reglas del ajedrez 5x5	11
5.1.3. Arquitectura basada en Transformers: Encoder y Self-Attention	14
5.1.4. Modelado del Aprendizaje por Refuerzo	14
5.2. Marco práctico	15
5.2.1. Codificación del Espacio de Estados (Input)	15
5.2.2. Codificación del Espacio de Acciones (Output)	16
5.2.3. Arquitectura Transformer	17
5.2.4. Algoritmo de Aprendizaje por Refuerzo: Proximal Policy Optimization (PPO)	20
5.2.5. Herramientas y tecnologías empleadas	22
5.2.6. Descripción del conjunto de datos (borrador)	22
5.2.7. Descripción del conjunto de datos	22
5.3. Diseño experimental	23
5.3.1. Diseño experimental para hipótesis 2	23
6. Planificación y seguimiento	26
7. Principales aportaciones	27
7.1. Resultados generales en el entorno de Minichess	27
7.2. Efectos de la representación sobre la eficiencia y calidad del aprendizaje	27
7.2.1. Evaluación en el Conjunto de Test Holdout y métricas utilizadas	27
7.2.2. Test de Significatividad Estadística	28
7.2.3. Evolución y Curvas de Aprendizaje	29
7.2.4. Enfrentamiento en Juego Real: Configuración y Resultados del Torneo	30
7.2.5. Discusión de los Resultados	30
7.2.6. Conclusión sobre la Hipótesis 2	33
7.3. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo	34
8. Conclusiones	34
9. Vías de trabajo futuro	34

Referencias	36
A. Anexos	38
A.1. Hiperparámetros del Entrenamiento y Estudio de Ablación	38
A.2. Formulación Matemática de las Funciones de Pérdida	40
A.2.1. Evolución de las Pérdidas por Cabeza durante el Entrenamiento	40
A.3. Resultados Detallados de los Tests de Significatividad Estadística	41
A.3.1. Test de Normalidad (Shapiro-Wilk)	41
A.3.2. Test de Diferencias Globales (One-Way ANOVA)	42
A.3.3. Comparaciones Múltiples por Parejas (Tukey HSD)	42

Índice de figuras

1.	Posición inicial del ajedrez Gardner.	12
2.	Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c... . . .	16
3.	Visualización de codificaciones de salida. De izquierda a derecha: a, b, c... . . .	17
4.	[Placeholder] Diagrama de la arquitectura utilizada (generado con IA, tiene errores / inconsistencias). Además, está desactualizado con respecto a la descripción anterior. Igual es mejor separar los diagramas de las cabezas, y hacerlos más abstractos en la arquitectura general.	19
5.	Distribución y variabilidad de las métricas en test sobre las 10 semillas independientes.	29
6.	Curvas de aprendizaje promediadas con sombreado de desviación estándar. . .	29
7.	Matrices de winrate cruzado del torneo de 100 partidas.	31
8.	Tasa de victoria y avance del ELO estimado a lo largo del entrenamiento. . . .	34
9.	Evolución de las pérdidas de entrenamiento desglosadas por cabeza de salida en el estudio de ablación (media de 10 semillas $\pm$ desviación estándar).	41

Índice de cuadros

1.	Rendimiento medio y desviación estándar de las métricas sobre el conjunto de test (10 semillas $\times$ 4 variantes).	28
2.	Clasificación final y Elo estimado de los agentes tras el torneo round-robin de 100 partidas.	31
3.	Comparativa de convergencia pre-entrenamiento vs tabula rasa (Hipótesis 3). . .	34
4.	Hiperparámetros fijos empleados en las ejecuciones del estudio de ablación de arquitectura.	39
5.	Comparación por parejas post-hoc de Tukey HSD para la exactitud del movimiento (Política).	42
6.	Comparación por parejas post-hoc de Tukey HSD para la exactitud del resultado (Valor).	42

1. Introducción

“Chess is a game of luck.
If you have a good opponent you have
bad luck, and if you have a bad
opponent you have good luck.”^[1]

Jacob Segal, 7 años

Este Trabajo de Fin de Grado consiste en la aplicación y estudio del Aprendizaje por Refuerzo (RL) sobre una variante de ajedrez simplificada, el ajedrez 5×5 (ajedrez de Gardner o Minichess). El objetivo central es validar el uso de arquitecturas Transformer en tareas de RL, estudiando el efecto de las representaciones del tablero y comprobando si el preentrenamiento supervisado mejora la convergencia del agente.

Dado que el ajedrez 8×8 demanda una capacidad computacional y un tiempo de ejecución que excede con creces los recursos disponibles en el marco del trabajo, se ha optado por utilizar el ajedrez 5×5 . Este entorno reducido mantiene las dinámicas esenciales del aprendizaje por refuerzo adversarial, como las acciones legales, las recompensas diferidas y la planificación estratégica, pero se mantiene computacionalmente manejable para la experimentación ágil.

El resto de este documento se estructura de la siguiente manera: en la Sección 2 se detallan los objetivos generales y específicos junto con el alcance del trabajo; en la Sección 3 se describe el estado del arte en transformers, aprendizaje por refuerzo y ajedrez y los antecedentes del problema; en la Sección 4 se introduce y resume la solución metodológica planteada.

En la Sección 5 se desarrolla el marco teórico (5.1) y práctico (5.2), detallando las hipótesis a probar, el modelado matemático del entorno de ajedrez, de la arquitectura transformer, la codificación de entradas y salidas y del problema de RL, además de justificar las herramientas utilizadas para el desarrollo. En la Sección 6 se muestra la planificación temporal del proyecto.

Las secciones finales presentan las aportaciones principales y resultados empíricos de los experimentos (Sección 7), las conclusiones y discusión entorno a los resultados (Sección 8) y las futuras líneas de investigación y limitaciones del trabajo (Sección 9). Los anexos contienen información adicional específica, o desarrollos / cuestiones que van mas allá del alcance del trabajo (Sección A).

2. Objetivos

2.1. Objetivo principal

Diseñar, entrenar y evaluar un agente de Inteligencia Artificial basado en arquitecturas Transformer mediante Aprendizaje por Refuerzo para el entorno de ajedrez 5×5 .

2.2. Objetivos específicos

1. Implementar un algoritmo de RL para ajedrez 5×5 , junto a la lógica necesaria para self-play.
2. Desarrollar y adaptar una arquitectura Transformer para procesar estados del tablero y predecir distribuciones de probabilidad sobre el espacio de acciones.
3. Evaluar empíricamente el impacto de distintas representaciones del tablero y del espacio de acciones en la velocidad de aprendizaje y rendimiento del modelo.
4. Comparar el rendimiento y la eficiencia de convergencia entre un agente entrenado mediante RL *tabula rasa* frente a un agente con inicialización mediante aprendizaje supervisado.

2.3. Alcance y limitaciones

El alcance de este trabajo se ha enmarcado en los siguientes límites prácticos e investigativos:

- **Entorno de juego:** Se limita estrictamente a la variante Gardner de ajedrez 5×5 . No se explorarán otras variantes del miniajedrez ni el ajedrez estándar 8×8 .
- **Recursos computacionales:** El entrenamiento de los modelos se realizará utilizando hardware de consumo accesible (una GPU de gama media/alta) con un límite temporal de entrenamiento acotado a un número de horas que permita la experimentación iterativa ágil.
- **Algoritmos de aprendizaje:** Se prioriza el uso de métodos de optimización basados en gradiente de política (PPO), evaluando el *Model-Free RL*. No se hará uso de algoritmos basados en planificación o búsqueda explícita (*Lookahead Search* como MCTS), con el fin de optimizar el tiempo de ejecución en hardware de consumo.

3. Antecedentes y contexto

El Aprendizaje por Refuerzo (RL) ha demostrado en los últimos años resultados extraordinarios en numerosos campos como el Go, el ajedrez, la síntesis de proteínas, los modelos de lenguaje y la robótica. Con la enorme capacidad de computación actual, que sigue creciendo día a día, el RL aprovecha las características enunciadas en *The Bitter Lesson* de Sutton^[2]: la búsqueda masiva y el aprendizaje de funciones a través del muestreo superan a largo plazo a los enfoques basados en conocimiento humano inyectado manualmente.

Ambos procesos, búsqueda y aprendizaje, son altamente exigentes en términos de computación, pero bajo la suposición de que esta aumenta al ritmo de la Ley de Moore, los métodos que aprovechan la computación superan eventualmente aquellos basados en conocimiento humano. Esto es especialmente cierto en entornos (discretos o continuos) con espacios de estado masivos, como son el ajedrez o la robótica.

3.1. Deep Reinforcement Learning en Ajedrez

Desde la irrupción del aprendizaje profundo y el aprendizaje por refuerzo en los juegos de tablero, se han explorado diversas arquitecturas neuronales. El hito inicial lo marcó AlphaGo^[3], que combinó redes convolucionales (CNNs) con búsqueda de árbol Monte Carlo (MCTS) para vencer al campeón mundial de Go. Posteriormente, esta metodología se generalizó con AlphaZero^[4], un algoritmo de RL capaz de dominar el ajedrez, el shogi y el Go partiendo desde cero (*tabula rasa*) y utilizando una única red neuronal para evaluar posiciones y priorizar jugadas. El impacto de estos métodos fué más allá, al demostrar el poder del *deep reinforcement learning* en otros dominios científicos complejos, como la predicción de estructuras de proteínas mediante AlphaFold^[5].

Paralelamente, en el ámbito de los motores de ajedrez tradicionales, enfoques como NNUE (*Efficiently Updatable Neural Networks*)^[6] tomaron un camino alternativo: en lugar de grandes redes convolucionales, emplean arquitecturas *feed-forward* muy ligeras con sesgos inductivos específicos del ajedrez, optimizadas para ejecutarse en CPU y priorizar la velocidad de evaluación y la profundidad de búsqueda táctica.

En estos motores de juego, las redes neuronales actúan como la parte *estratégica* mientras que la búsqueda actúa como la parte *táctica*, decidiendo a corto plazo bajo una serie de restricciones más claras (jugadas forzadas, amenazas claras como jaques o pérdida de material), y depende más del *cálculo*. Esto es, en cierto modo, reminiscente de cómo los humanos estructuramos nuestro pensamiento a la hora de jugar al ajedrez.

3.2. Transformers en ajedrez

Entre los enfoques anteriores, ninguno hacía uso de mecanismos de atención ni de arquitecturas basadas en Transformers, lo que nos lleva a explorar el potencial de estos últimos. Más recientemente, tras el éxito de esta arquitectura en los modelos de lenguaje (LLMs), han surgido aproximaciones que buscan aplicarla a tareas de RL y ajedrez. En este trabajo, se busca demostrar que la arquitectura Transformer es altamente competitiva para la tarea de RL, utilizando un entorno 5×5 para facilitar la simulación y el muestreo iterativo.

En este ámbito, Ruoss et al.^[7] demostraron la viabilidad de realizar planificación amortizada utilizando Transformers a gran escala aplicados al ajedrez. Por otro lado, Monroe y Chalmers^[8] desarrollaron un modelo basado en Transformers para dominar el ajedrez, sentando las bases para Chessformer, una arquitectura unificada propuesta posteriormente por Monroe et al.^[9]. Además, Jenner et al.^[10] aportaron evidencia sobre la existencia de capacidades de anticipación y planificación (*look-ahead*) aprendidas de forma implícita por este tipo de redes, mientras que el proyecto open source Leela Chess Zero también ha incorporado y evaluado estas arquitecturas en su motor de juego^[11].

Por su parte, el estudio del miniajedrez o ajedrez en tableros reducidos tiene una larga trayectoria. Existen miles de variantes del ajedrez; la variante de Gardner fue propuesta originalmente por Martin Gardner en 1969^[12]. Este tipo de entornos se ha empleado tradicionalmente para analizar la complejidad matemática del juego y estudiar algoritmos de búsqueda heurística, al reducir drásticamente el espacio de estados sin perder la naturaleza del ajedrez convencional; necesidad de táctica y estrategia, y emergencia de dinámicas interesantes al contar con distintos tipos de pieza (lo que aumenta el número de combinaciones posibles). El ajedrez 5×5 es particularmente interesante ya que permite mantener todos los tipos de pieza (torre, alfil, caballo, dama y rey) y una disposición inicial similar a la del ajedrez estándar.

En 2013, Mehdi y Prost resolvieron débilmente el ajedrez Gardner, esto es, obtuvieron un algoritmo que permite asegurar el resultado de la partida partiendo de la posición inicial, pero no desde cualquier posición arbitraria. Utilizando técnicas de búsqueda e intervención humana, obtuvieron un oráculo para el jugador blanco y negro que asegura un empate partiendo de la jugada inicial.^[13]

4. Resumen de la solución propuesta

[TODO: Redactar bien el pipeline, por ahora solo es un esqueleto. Como es un resumen, tampoco tiene que ser super extenso, un par de líneas por cada parte quizás. Btw, no se solapa algo con la planificación?]

1. Generación de un dataset de partidas de Minichess con un motor clásico.
2. Estudio, filtrado y procesado de datos.
3. Diseño de arquitectura de un modelo Transformer Encoder.
4. Evaluación de distintos sesgos de input / output en fase de entrenamiento supervisado.
5. Definición del bucle de entrenamiento e implementación de algoritmo de RL (PPO).
6. Fase de evaluación contra una *baseline* (ej. Stockfish o heurísticas simples).
7. Evaluación de red preentrenada VS. from-scratch (mismo numero de epochs).

5. Marco teórico y práctico. Herramientas empleadas

5.1. Marco teórico

5.1.1. Diseño Experimental e Hipótesis

Para dar respuesta a los objetivos planteados, el desarrollo práctico se articula en torno a la demostración de las siguientes hipótesis experimentales, las cuales estructuran el diseño experimental del trabajo:

- **Hipótesis 1 (Reducción del problema):** El ajedrez 5x5 como entorno simplificado de RL puede conservar suficientes dinámicas esenciales del aprendizaje por refuerzo adversarial al tiempo que se mantiene computacionalmente manejable para la experimentación.
- **Hipótesis 2 (Representación):** El uso de representaciones estructuradas del tablero y del espacio de acciones (ej. planos espaciales vs. predicción por casillas origen/destino) mejora la eficiencia del aprendizaje frente a representaciones planas.
- **Hipótesis 3 (Inicialización):** Un agente inicializado mediante aprendizaje supervisado a partir de jugadas generadas por un motor de referencia alcanza un nivel de rendimiento competitivo en menos *epochs* que un agente entrenado de forma puramente *tabula rasa*.

5.1.2. Entorno: Reglas del ajedrez 5x5

El ajedrez 5×5 en su variante de Gardner es una reducción matemática del ajedrez estándar. Las reglas básicas de movimiento y captura para cada pieza individual son idénticas a las del ajedrez estándar 8×8 , con las siguientes modificaciones estructurales:

- **Tablero:** tamaño de 5×5 casillas (columnas *a* a *e*, filas 1 a 5).
- **Disposición inicial:** La misma que el ajedrez estándar, pero con las piezas del lado de rey eliminadas, y una sola fila de separación entre peones. Cada jugador cuenta con 10 piezas. En la fila 1 (blancas) y fila 5 (negras) se colocan las piezas mayores en el orden: Torre (*R*), Caballo (*N*), Alfil (*B*), Dama (*Q*) y Rey (*K*). Los peones correspondientes se colocan en la fila 2 (blancas) y fila 4 (negras).
- **Omisión de reglas especiales:** No existe el enroque ni el avance doble de peón. Como consecuencia directa, tampoco existe la captura al paso (*en passant*).
- **Promoción de peones:** Cuando un peón alcanza la fila límite opuesta (fila 5 para blancas, fila 1 para negras), es obligatorio promocionarlo a una de las siguientes cuatro piezas: Dama, Torre, Alfil o Caballo.
- **Condiciones de fin de partida:** La partida finaliza por (A) jaque mate (el rey está amenazado y no puede retirarse a una casilla a salvo, cubrirse, o capturar a la pieza que lo amenaza), (B) rey ahogado (*stalemate*, el rey no está en jaque pero el jugador no tiene ningún movimiento legal), (C) insuficiencia de material para dar jaque mate (posición muerta), (D) si pasan 75 movimientos sin capturas ni avances de peón, o (E) tablas por quintuple repetición (cinco posiciones iguales a lo largo de toda la partida (no necesariamente consecutivas)).

- **Condiciones reclamables**: la regla de las 5 repeticiones y de los 75 movimientos tiene sus contrapartes “*reclamables*”: la regla de las 3 repeticiones y la de los 50 movimientos requieren que uno de los jugadores reclame tablas para activarse.

Esta variante mantiene la naturaleza estratégica y la profundidad del ajedrez, pero en una fracción de su complejidad de cálculo original.

Representación de estados mediante notación FEN

Para interactuar con el entorno de ajedrez y alimentar los modelos de aprendizaje, se utiliza la notación FEN (*Forsyth-Edwards Notation*). Esta notación describe una posición particular del tablero mediante una cadena de texto que codifica la ubicación de las piezas, el jugador al que le toca mover y las disponibilidades de reglas especiales. En el contexto del ajedrez 5x5, el FEN se simplifica ya que cierta información no es aplicable a la variante de Gardner (como los enroques o las capturas *en passant*).

El FEN inicial de la variante Gardner se representa como:

`rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1`

La Figura 1 ilustra la disposición física y la colocación de las piezas en el tablero Gardner al inicio de la partida.



Figura 1: Posición inicial del ajedrez Gardner.

El ajedrez como juego y proceso teórico de decisión

El ajedrez es un juego de información perfecta y completa. En todo momento los jugadores tienen información total acerca de las funciones de utilidad de su rival y del estado de la partida, tanto presente como pasado. Esto proporciona fundamentos teóricos sólidos para probar algoritmos de RL; un entorno determinista y con funciones de transición bien definidas.

Dado el factor de ramificación estimado del ajedrez estándar (media de 35 hijos por nodo) y la complejidad del árbol de juego (10^{120}), propuesta por Claude Shannon en su artículo fundacional^[14], la variante de ajedrez 5×5 se establece como un entorno matemáticamente equivalente en sus reglas de transición, pero con un espacio de estados y una complejidad del árbol de juego acotados.

Para estimar la complejidad de la variante Gardner 5×5 , siguiendo un razonamiento similar al de Shannon, se pueden calcular las siguientes métricas:

- **Espacio de estados $\mathcal{S}$** : Mehdi y Prost estimaron que el número total de posiciones legales en el ajedrez 5×5 es aproximadamente 9×10^{18} ^[13], descontando posiciones

ilegales (reyes en jaque simultáneo, peones en las filas de inicio/promoción, etc...).

TODO: no mencionan como lo obtuvieron. puede que sea un recuento combinatorio estilo total = permutaciones(25, 10) * comb(15,5) * comb(10,5), (ordenar las 10 piezas principales, luego peones negros y luego blancos), pero esto deja muchas jugadas que son ilegales o que simplemente no tienen sentido , ademas de no contar las posiciones con capturas.

- **Complejidad del árbol de juego:** Considerando un factor de ramificación promedio $b \approx 12$ a 15 jugadas legales por posición y una profundidad promedio de partida $d \approx 20$ plies (medios movimientos),

(TODO:por ahora estos 3 numeros son placeholders. son estimaciones sensatas pero estaría bien sacar datos exactos de las estadísticas de generación de partidas)

la complejidad de Shannon b^d para Gardner es:

$$N_{upper} \approx 15^{20} \approx 3,3 \times 10^{23}$$

$$N_{lower} \approx 12^{20} \approx 3,8 \times 10^{21}$$

Estos valores son infinitamente menores que el límite superior del ajedrez estándar (10^{120}), lo que permite que el *self-play* converja con órdenes de magnitud menos recursos computacionales.

Dada la complejidad del entorno, nos gustaría poder modelarlo de modo que el aprendizaje se realice únicamente sobre el estado actual, ya que comprobar el historial de jugadas completo tendría un coste computacional excesivo. Buscamos que la probabilidad de alcanzar un estado futuro dependa únicamente del estado presente, y no del historial de estados anteriores. Esta es la propiedad de Markov.

El ajedrez como Proceso de Decisión de Markov (MDP)

Un proceso estocástico cumple la propiedad de Markov si la probabilidad de alcanzar un estado futuro, s_{t+1} , depende únicamente del estado presente s_t , siendo condicionalmente independiente del historial de estados anteriores. Formalmente:

$$P(s_{t+1}|s_t, s_{t-1}, \dots, s_0) = P(s_{t+1}|s_t)$$

Si definimos el estado del ajedrez únicamente por la disposición física de las piezas en el tablero temos que $s_t = B_t$, entonces el juego no es estrictamente un MDP. Recordemos las reglas mencionadas anteriormente: la triple repetición y la regla de los 50 movimientos tienen en cuenta el historial de estados, no solo el actual.

Para modelar el ajedrez 5×5 como un MDP, es necesario expandir el espacio de estados. Definimos el nuevo estado expandido $s \in \mathcal{S}$ como una tupla $s = (B, p, h)$ donde:

1. B : La disposición exacta de todas las piezas en el tablero de 5×5 en el instante t .
2. p : Un reloj o contador (*half-move clock*) que registra el número de medios movimientos realizados desde la última captura o avance de peón.

3. h : Un registro de las posiciones previas ocurridas desde la última transición irreversible (o, de forma computacionalmente más eficiente, un mapa de frecuencias que contabilice cuántas veces se ha visitado la disposición B actual en la partida en curso).

Al encapsular explícitamente los contadores p y h dentro de la representación del estado actual s_t , el modelo no necesita consultar cronológicamente cómo se llegó a dicho estado. Toda la información temporal y causal necesaria para evaluar la legalidad de las jugadas futuras o detectar empates está autocontenida en el instante presente. Bajo esta formulación, se satisface la propiedad de Markov, lo que nos permite simplificar el modelado.

5.1.3. Arquitectura basada en Transformers: Encoder y Self-Attention

A diferencia de los modelos de lenguaje (LLMs) que generan secuencias texto *token a token* utilizando una arquitectura de decodificador con enmascaramiento causal, la evaluación de un tablero de ajedrez requiere una comprensión bidireccional del estado completo en un instante de tiempo / posición. No es necesario enmascarar la atención para evitar observar tokens futuros, ya que por la propiedad de Markov demostrada anteriormente, una posición no depende de estados posteriores ni anteriores.

Por ello, la arquitectura propuesta se fundamenta en un **Transformer Encoder**. Este modelo procesa simultáneamente todas las casillas del tablero, permitiendo que cada pieza calcule su relevancia respecto a todas las demás sin restricciones de enmascaramiento. El núcleo de esta arquitectura es el mecanismo de Auto-Atención (*Self-Attention*)^[15], definido formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Donde Q (Query), K (Key) y V (Value) son matrices obtenidas mediante proyecciones lineales de las representaciones del tablero. Dada una entrada X ,

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

W^Q, W^K, W^V son matrices de pesos entrenables, y d_k representa la dimensión de las claves, el cual puee variar entre configuraciones de arquitectura. Este mecanismo se replica en cada una de las h cabezas de atención, permitiendo capturar distintas relaciones entre las piezas. El resultado se concatena y se proyecta linealmente de vuelta a la dimensión original de la entrada.

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, W^O es una matriz de pesos entrenables y h es el número de cabezas de atención.

5.1.4. Modelado del Aprendizaje por Refuerzo

El problema del ajedrez 5×5 se modela a través de una formulación clásica de Proceso de Decisión de Markov (MDP). En cada paso t , el agente observa el estado actual s_t , elige una acción estocástica basada en una política parametrizada $\pi_\theta(a_t|s_t)$ y pasa a un nuevo estado

s_{t+1} . La iteración prosigue hasta el final de la trayectoria (fin de la partida), momento en el cual el agente interactúa con el entorno recibiendo una recompensa terminal (victoria, derrota o empate).

A diferencia de enfoques como MCTS que optimizan una pérdida de entropía cruzada mediante búsqueda explícita, este trabajo emplea una red neuronal de política y valor conjunta sin simulación del futuro. La red se optimiza directamente utilizando la *clipped surrogate objective function* de Proximal Policy Optimization^[16]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right]$$

donde $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ es el cociente de probabilidades, $\hat{A}_t$ es la estimación de la ventaja y ϵ es el hiperparámetro de recorte (clipping). La red también minimiza el error cuadrático medio en la estimación del valor $V_\theta(s_t)$. La función de pérdida conjunta total es:

$$\mathcal{L} = -L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$$

Diseño de la Función de Recompensa

El Aprendizaje por Refuerzo es altamente susceptible al fenómeno de *reward hacking*, donde el agente aprende a explotar lagunas en la función de recompensa para maximizar su puntuación sin alcanzar el objetivo real del problema. Por ejemplo, si se otorgara una recompensa positiva por cada captura de material, el agente podría aprender a retrasar el jaque mate indefinidamente para “farmear” piezas del oponente.

Para evitar este comportamiento subóptimo, se define una función de recompensa dispersa (*sparse reward*) R_t , que únicamente emite una señal al alcanzar un estado terminal final, denominado T :

$$R_t = \begin{cases} 1 & \text{si el estado } T \text{ es victoria} \\ -1 & \text{si el estado } T \text{ es derrota} \\ 0 & \text{si el estado } T \text{ es empate o no terminal} \end{cases}$$

Esta escasez de señal dificulta las fases iniciales del entrenamiento, justificando así la necesidad de técnicas avanzadas o inicialización supervisada para guiar al modelo en las etapas tempranas para acelerar la convergencia de PPO.

5.2. Marco práctico

5.2.1. Codificación del Espacio de Estados (Input)

Para evaluar la Hipótesis 1, se instrumentarán y compararán dos representaciones del tablero de ajedrez 5x5, analizando su impacto en la asimilación espacial del modelo Transformer.

- **Representación plana simple (Byte Array):** El tablero se representa como un vector unidimensional de 27 *bytes* (índices 0-24), correspondientes a las 25 casillas del tablero de Minichess. Cada casilla almacena un número entero que identifica unívocamente una de las 12 piezas posibles (6 blancas y 6 negras) o el estado de casilla vacía. Adicionalmente se almacenan el número de repeticiones y el número de jugadas sin capturas / jaques en los 2 bytes finales. Esta representación es minimalista pero

computacionalmente eficiente, y delega en el Transformer la responsabilidad de deducir la geometría 2D del tablero.

- **Representación espacial (estilo AlphaZero):** El tablero se expande en un tensor multicanal de dimensiones $C \times 5 \times 5$. Cada canal C está asignado a un tipo de pieza por color y actúa como un plano binario indicando su presencia o ausencia. Esta codificación facilita el reconocimiento de patrones espaciales a través de sesgos inductivos explícitos. Se añaden, además, canales binarios adicionales para variables de estado globales como el jugador al que le toca mover y los límites del half-move clock.

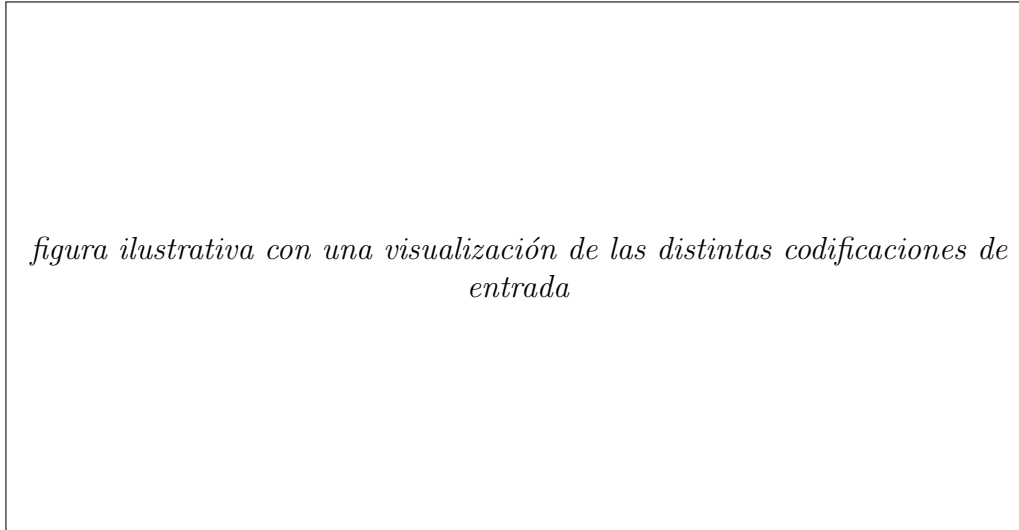


figura ilustrativa con una visualización de las distintas codificaciones de entrada

Figura 2: Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c...

5.2.2. Codificación del Espacio de Acciones (Output)

De manera análoga, la forma en que la red proyecta sus predicciones altera drásticamente la topología del espacio de optimización. Se implementarán las siguientes cabezas de salida:

- **Política Plana Discreta (Flat Policy):** Consiste en un vector de salida lineal que mapea todas las posibles transiciones legales del juego. En un tablero 5x5, considerando que una pieza no puede moverse a su propia casilla origen, existen $5^2 \times (5^2 - 1) = 600$ movimientos posicionales puros. Adicionalmente, las promociones de peón requieren un modelado independiente.

En Minichess, los peones en las columnas exteriores (a, e) tienen 2 opciones al coronar (avance o captura simple), mientras que los peones en columnas centrales (b, c, d) tienen 3. Esto resulta en $(2 + 3 + 3 + 3 + 2) = 13$ movimientos de promoción. Multiplicado por las 4 piezas posibles de promoción (Reina, Torre, Alfil, Caballo) y los 2 colores, se añaden 104 neuronas. La salida de la política (*policy head*) será, por tanto, un vector estricto de **704 neuronas**.

- **Política Factorizada (Origen-Destino) como Tarea Auxiliar:** Esta codificación no sustituye a la política discreta plana principal de 704 salidas; añade una cabeza auxiliar para informar más del gradiente de pérdida durante el entrenamiento. Se compone de tres cabezas complementarias: una para la casilla de origen del movimiento (25 salidas), otra para la casilla de destino (25 salidas) y una tercera para predecir la clase de promoción (9 salidas). La pérdida de estas cabezas se pondera con un factor

de 0.5 y se suma a la pérdida total. Esto reduce el espacio de salida de esta cabeza de 704 a 59 dimensiones en total, simplificando la tarea de optimización del gradiente. El objetivo es facilitar el aprendizaje de las representaciones del backbone de atención, y acelerar la convergencia inicial del entrenamiento proporcionando una señal de error más explícita y estructurada.

- **Cabeza de valor:** Se incluye una cabeza de valor que predice el valor esperado de la posición actual, lo que permite al modelo aprender a evaluar la calidad de las posiciones además de predecir acciones. Esta cabeza se implementa como una proyección lineal seguida de una activación tanh para acotar la salida en el rango $[-1, 1]$, donde -1 representa una derrota segura, 0 un empate y +1 una victoria segura.

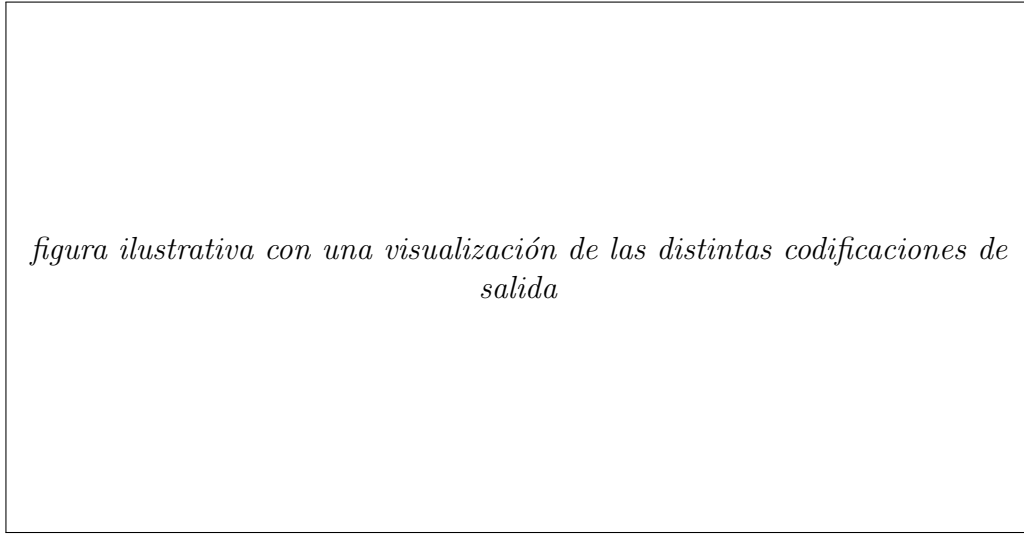


Figura 3: Visualización de codificaciones de salida. De izquierda a derecha: a, b, c...

5.2.3. Arquitectura Transformer

La arquitectura neural propuesta se basa en un modelo **Transformer Encoder** que procesa el estado del tablero de forma bidireccional, permitiendo que cada componente preste atención a todos los demás, y se describe a continuación:

1. **Capa de Proyección de Entrada:** Dependiendo de la representación seleccionada (de las mencionadas en la sección 5.2.1):
 - *Para la representación plana simple:* Consta de 25 casillas (que clasifican las piezas o casilla vacía), más la cuenta de repeticiones y la regla de los 50 movimientos (27 elementos en total). Adicionalmente, se concatena un **token de clase CLS**, resultando en una secuencia de 28 tokens que se proyectan a la dimensión d_k mediante una tabla de embeddings.
 - *Para la representación espacial (estilo AlphaZero):* El tablero reconstruye un tensor de dimensiones $15 \times 5 \times 5$ (12 canales de piezas, 1 canal para el jugador activo, 1 canal de repeticiones y 1 canal de halfmove clock). Cada una de las 25 casillas del tablero cuenta con un vector de 15 canales que se proyecta linealmente a la dimensión d_k . Adicionalmente, se concatena el **token de clase CLS**, resultando en una secuencia de 26 tokens.
2. **Codificación Posicional:**

- *Para la representación plana simple:* Se añade un *embedding* posicional absoluto `w_pos_embed` unidimensional de tamaño 28, asignando un vector aprendido a cada posición de la secuencia.
 - *Para la representación espacial:* Se inyecta información geométrica explícita en 2D. Se definen dos tablas de embeddings aprendidas: `row_pos_embed` de tamaño (5, D) (para las filas) y `col_pos_embed` de tamaño (5, D) (para las columnas). Para cada una de las 25 posiciones del tablero en la coordenada (y, x) , su codificación posicional es la suma de los embeddings de su fila y su columna. Para el token CLS se concatena un parámetro aprendido independiente `cls_pos_embed`.
3. **Pila de Bloques Encoder:** El tensor resultante se procesa a través de una serie de L capas de codificación con h cabezas. Cada capa contiene:
- Un mecanismo (*Multi-Head Self-Attention*) sin causal-masking.
 - Conexiones residuales de inicio a fin y normalización, utilizando *pre-norm* y **RMS-Norm**. La convención *pre-norm* aplica normalización antes de cada subcapa de la forma $z = x + \text{sublayer}(\text{normalization}(x))$, donde *sublayer* es una de las L capas encoder, mejorando la estabilidad del entrenamiento^[17]. La normalización RMS es utilizada por transformers modernos debido a que reduce la complejidad computacional frente a alternativas como LayerNorm^[18].
 - Una capa *Feed-Forward Network* o *Multi-Layer Perceptron* (MLP), compuesta de la forma:

$$\text{FFN}(x) := FC_{\text{expand}}(x) \rightarrow \text{GELU} \rightarrow FC_{\text{project}}(x) \rightarrow \text{Dropout}$$

4. **Cabezas de Salida:** Tras la última normalización RMS, el estado se divide, extrayendo el tensor del *CLS token* y aislando los 25 vectores resultantes del estado final de las iteraciones de atención sobre las celdas del tablero. A continuación se bifurca en los siguientes procesos:
- **Cabeza de Política Orientada a Relaciones (“*Matrix Policy Head*”):** Proyecta cada uno de los 25 *tokens* de las casillas paralelamente evaluando si estas funcionan como origen (`proj_from`) y como destino (`proj_to`) mediante MLPs. A continuación realiza un producto interno espacial (*batched matmul*) entre estas incrustaciones elaborando una matriz de correlación 25×25 . Paralelamente, se obtienen otros 104 *logits* del tensor *CLS* que reflejan opciones de promoción, y se concatenan a los 600 movimientos ordinarios. Obtenemos por lo tanto, una distribución de 704 *acciones*.
 - **Cabeza de Valor (*Value Head*):** Procesa únicamente el *token CLS* final en un *MLP* denso (Expansión Lineal, GELU, Dropout, Reducción de dim a escalar, Tanh) que acota la salida en el intervalo $[-1, 1]$.
 - **Cabezas Auxiliares de Política Factorizada:** Estas cabezas no se activan siempre. Se trata de las cabezas mencionada en la sección 5.2.2. Decodifican las representaciones del tablero y de CLS mediante tres proyecciones independientes en logits de casilla de origen (25), destino (25) y tipo de promoción (9). Estas cabezas se evalúan únicamente durante el entrenamiento supervisado, con propósitos de mejora de la señal de error.

5. **Funciones de Pérdida combinada:** Durante el entrenamiento supervisado se minimiza una función de pérdida multiobjetivo. Para las variantes sin pérdida factorizada auxiliar, la pérdida total $\mathcal{L}$ es la suma directa de la pérdida de la política principal $\mathcal{L}_p$ (entropía cruzada con logits), y la pérdida de valor, $\mathcal{L}_v$ (error cuadrático medio):

$$\mathcal{L} = \mathcal{L}_p + \mathcal{L}_v \quad (1)$$

Para las variantes con pérdida factorizada auxiliar, se añade la pérdida de la cabeza auxiliar factorizada, $\mathcal{L}_{aux}$ (entropía cruzada), escalada por un factor de regularización de 0,5 para informar mejor al gradiente sin eclipsar la política principal:

$$\mathcal{L} = \mathcal{L}_p + \mathcal{L}_v + 0,5 \cdot \mathcal{L}_{aux} \quad (2)$$

Las formulaciones exactas de cada componente de pérdida se detallan en el Anexo A.2.

La figura 4 ilustra la arquitectura propuesta, mostrando el flujo de datos desde la entrada de la representación hasta las salidas de las cabezas de política y valor.

Bidirectional Encoder Minichess Transformer (BEMT)

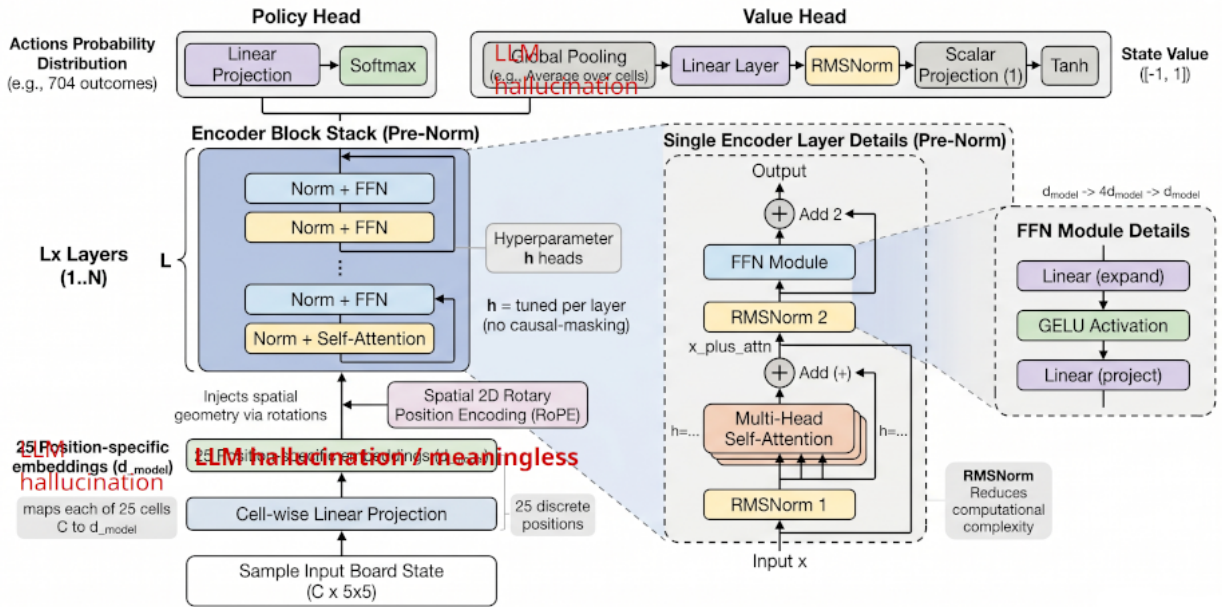


Figura 4: [Placeholder] Diagrama de la arquitectura utilizada (generado con IA, tiene errores / inconsistencias). Además, está desactualizado con respecto a la descripción anterior. Igual es mejor separar los diagramas de las cabezas, y hacerlos más abstractos en la arquitectura general.

5.2.4. Algoritmo de Aprendizaje por Refuerzo: Proximal Policy Optimization (PPO)

Cabe preguntarse: ¿por qué este algoritmo en lugar del MCTS usado en enfoques como AlphaGO o AlphaZero? Clásicamente, metodologías como MCTS y *Expert Iteration* actúan más como planificadores a futuro que como RL propiamente dicho: descubren futuros hipotéticos mediante búsqueda por fuerza bruta al contar con un modelo perfecto del entorno. En ellos, la red neuronal actúa como una memoria supervisada que aproxima las probabilidades de visita ya descubiertas por la búsqueda. Estos sistemas no están aprendiendo *per se* (algo que se obtiene de la experiencia, y por tanto del pasado), sino que están planificando (algo que se obtiene de la previsión del futuro, y por tanto de la simulación, el futuro).

Por el contrario, Proximal Policy Optimization (PPO) es un algoritmo de aprendizaje por refuerzo puro (*Model-Free*). La red neuronal carece de un mecanismo explícito de previsión del futuro mediante un árbol con estados; es ella, iterativamente a través métodos Actor-Crítico y puramente por descenso de gradiente, la que asimila las estrategias óptimas guiada por ensayo y error. Además, al requerirse exactamente una única inferencia por tablero en cada estado, los tableros / estados pueden agruparse en (*batches*) mayores, maximizando la ocupación de GPU.

Mecánica de Optimización y Estimación de Ventaja

El algoritmo PPO actualiza los pesos para evitar colapsar debido a cambios demasiado severos de la política por una actualización errónea. Esta es la función fundamental del hiperparámetro de recorte ϵ (*clipping mechanism*). Este asegura que la actualización no destruya la política existente, al penalizar las actualizaciones estocásticas grandes que diverjan significativamente de la política anterior empleada para recolectar el episodio en cuestión.

Para balancear correctamente el sesgo y la varianza a la hora de determinar lo buena que es la acción tomada respecto a la acción promedio en el estado actual, se emplea la Estimación Generalizada de Ventaja (GAE - *Generalized Advantage Estimation*)^[16]. El algoritmo se puede sintetizar en el pseudocódigo del Algoritmo 1.

TODO revisar comparar con https://spinningup.openai.com/en/latest/algorithms/ppo.html#spinup.ppo_pytorch y [paper original](#)

Algorithm 1 Proximal Policy Optimization (PPO-Clip)

```

1: Inicializar parámetros de política y valor  $\theta$ 
2: for iteración  $i = 1, 2, \dots$  do
3:   Ejecutar política parametrizada  $\pi_{\theta_{old}}$  en  $N$  entornos paralelos durante  $T$  pasos tem-
      porales
4:   Calcular las estimaciones de ventaja  $\hat{A}_t$  mediante GAE, en base al valor de  $V_\theta(s_t)$  y
      las recompensas obtenidas
5:   for época  $k = 1, \dots, K$  do
6:     for cada mini-batch do
7:       Calcular el cociente  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 
8:       Calcular pérdida recortada:  $L^{CLIP} \leftarrow \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$ 
9:       Calcular pérdida de valor:  $L^{VF} \leftarrow (v_\theta(s_t) - V_t^{target})^2$ 
10:      Calcular pérdida de entropía:  $L^{ENT} \leftarrow H(\pi_\theta(s_t))$ 
11:      Retropropagar para minimizar  $L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$ 
12:     end for
13:   end for
14:    $\theta_{old} \leftarrow \theta$ 
15: end for
    
```

En este pseudocódigo, se deben denotar varios aspectos:

- Tanto V_θ como π_θ están parametrizadas por el mismo θ , dado que ambas funciones comparten el backbone de la arquitectura, pero cada una se predice en una de sus cabezas.
- c_1 y c_2 son coeficientes de ponderación para la pérdida de valor y la pérdida de entropía, respectivamente. La función de entropía $H(\pi_\theta(s_t))$ se incluye para fomentar la exploración durante el entrenamiento, y el valor objetivo V_t^{target} se calcula utilizando las recompensas obtenidas y las estimaciones de valor futuro.
- La ventaja en el pseudocódigo anterior se calcula utilizando GAE, y se controla mediante el hiperparámetro λ que ajusta el sesgo-varianza de la estimación. Un valor de λ cercano a 1 favorece una estimación de ventaja con bajo sesgo pero alta varianza (aproxima Monte Carlo), mientras que un valor cercano a 0 produce una estimación con alto sesgo pero baja varianza que aproxima *TD-learning*.

Este hiperparámetro no está presente explícitamente en el pseudocódigo, pero se utiliza en la función de cálculo de $\hat{A}_t$ para determinar cómo se combinan las recompensas futuras y las estimaciones de valor para obtener una ventaja más estable y eficiente. El cálculo de la ventaja $\hat{A}_t$ mediante GAE se realiza utilizando la siguiente fórmula:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$$

5.2.5. Herramientas y tecnologías empleadas

El desarrollo práctico e investigativo del proyecto se implementará utilizando las siguientes herramientas de desarrollo de software libre:

- **Lenguaje de programación:** Python 3.12+ por su extenso soporte y bibliotecas nativas de Deep Learning.
- **Framework de Deep Learning:** PyTorch para el diseño de redes neuronales, cálculo automático de gradientes y aceleración en GPU.
- **Motor de Ajedrez:** Se emplea **Fairy-Stockfish**, una versión de Stockfish para variantes personalizadas, a través de la API **pyffish** de Python y a través de su CLI propia para generación de datos para la fase de aprendizaje supervisado. Este motor proporciona: (a) validación robusta y rápida de la legalidad de movimientos y reglas de la variante Gardner 5×5 , y (b) evaluaciones heurísticas adecuadas para inicializar la red mediante aprendizaje supervisado.
- **Seguimiento de experimentos:** **tensorboard** se utilizó para la visualización en tiempo real del progreso de entrenamiento, entropía de la política, pérdida multiobjetivo y curvas de convergencia.
- **Reproducibilidad, dependencias y manejo de módulos en Python:** **uv** para la gestión de entornos virtuales y dependencias, asegurando la portabilidad y reproducibilidad del código (**uv** permite generar archivos de requisitos específicos y fijar una versión de Python).

5.2.6. Descripción del conjunto de datos (borrador)

TODO: poner una muestra, explicar modificaciones (mascaras añadidas p ej), estadísticas en el anexo (con imagenes...)

- las estadísticas más importantes ponerlas aquí (ratio draws) - explicar importancia de distintas profundidades. DECIDIR DEFINITIVAMENTE si el aprendizaje supervisado final lo voy a hacer con una mezcla o con uno solo. update: finalmente lo estoy haciendo solo con la versión de depth 4. - hacer que jueguen entre ellos distintos modelos a distintas profundidades en caso de que use varias. update: en ppi esto ya no es viable - explicar peculiaridades: quiescent positions, filtrado de posiciones repetidas, filtrado de posiciones tácticas puras. explicar lo de las coronaciones que solo son a una dama, y el ratio de victorias por color... es importante aclarar que son datos complejos: modelar búsqueda de profundidad 4 y además con el añadido de filtrar posiciones tácticas simples (capturas, movimientos excesivamente forzados, etc) lleva a la red a tener que internalizar o patrones muy complejos o la idea de búsqueda en profundidad. además de tener que hacer esto con las recompensas dispersas y con poca densidad informativa que son la victoria / derrota / empate - para esto habrá bastante información seguramente en las entradas de blog que escribí

5.2.7. Descripción del conjunto de datos

TODO explicar como se ve el conjunto de datos, como es una instancia... los datos que importan y los que no... menos slop.

Para el entrenamiento en la fase de aprendizaje supervisado, se ha optado por utilizar exclusivamente un conjunto de datos sintéticos generado con una profundidad de búsqueda de 4

jugadas **fairy-stockfish**, descartando por completo el uso de mezclas con evaluaciones de profundidades inferiores (D2 o D3). Este conjunto de datos definitivo cuenta con un volumen total de 60,030,000 posiciones, el cual ha sido procesado mediante una división estricta de un 97 % para el conjunto de entrenamiento (58,234,050 posiciones) y un 3 % reservado para el conjunto de validación (1,795,950 posiciones).

La elección de entrenar el modelo únicamente con datos analizados a profundidad 4 responde a la necesidad de capturar la complejidad intrínseca del juego posicional y estratégico en el tablero reducido de Gardner 5×5 . Modelar el comportamiento de un motor clásico a este nivel obliga a la arquitectura a ir más allá de heurísticas simples. El conjunto de datos presenta cierta complejidad debido a dos factores principales:

- **Aislamiento de posiciones quiescentes (*quiescent positions*):** Se han filtrado y eliminado sistemáticamente las posiciones tácticas puras (tales como capturas inmediatas de material o movimientos bajo jaques excesivamente forzados). Este filtrado es común en redes neuronales para ajedrez que se utilizan en conjunto con algún tipo de algoritmo de búsqueda, ya que garantiza que el Transformer no aprenda respuestas reactivas simples, sino que se vea obligado a internalizar patrones posicionales complejos o a emular conceptualmente la propia idea de búsqueda en profundidad bajo condiciones de recompensas totalmente dispersas y de baja densidad informativa (las señales terminales de victoria, derrota o empate).
- **Filtrado de posiciones repetidas:** Se ha aplicado un proceso de deduplicación sobre el archivo crudo para eliminar estados idénticos del tablero, previniendo memorización y posible contaminación de los conjuntos de validación o test.

Asimismo, el conjunto de datos cuenta con particularidades estructurales en el espacio de acciones e indicadores de balance del juego:

- **Restricción en las coronaciones:** A diferencia del modelo teórico que contempla opciones múltiples de promoción, los movimientos de coronación registrados en este conjunto de datos se encuentran limitados exclusivamente a la dama, debido a una limitación en la librería de generación de datos. En la práctica, esta suele ser la mejor jugada disponible, y no representa un sacrificio de calidad excesivo. La codificación de salida de la red neuronal sigue permitiendo la sub-coronación, en caso de ser necesaria en la fase de RL.
- **Métricas de distribución y balance:** El análisis estadístico del conjunto de posiciones refleja las propiedades simétricas y el margen de ventaja del entorno analizado. Más estadísticas acerca del conjunto de datos pueden encontrarse en el anexo. **[PLACEHOLDER: Insertar aquí el ratio exacto de tablas (draws ratio) y la proporción o porcentaje de victorias por color (blancas vs. negras) extraídos de tus scripts de análisis].**

5.3. Diseño experimental

5.3.1. Diseño experimental para hipótesis 2

Para demostrar el efecto de distintas representaciones del input y de output, se ha llevado a cabo el siguiente experimento:

1. **Fijar una “receta” de red neuronal y de entrenamiento:** hiperparámetros de la red como la dimension de embeddings o bloques transformer + hiperparámetros de

- entrenamiento como LR, batch size, epochs y parametros de AdamW (detalles en 4)
2. **Definimos 4 versiones:** 1 representa al modelo base y las 3 restantes, ablaciones sobre este. Las 4 versiones son:
 - (Baseline) Codificación de input simple. Cabezas simples: política y valor.
 - Codificación de input simple. Cabezas factorizadas: política, valor y política factorizada.
 - Codificación de input bidimensional (estilo AlphaZero). Cabezas simples: política y valor.
 - Codificación de input bidimensional (estilo AlphaZero). Cabezas factorizadas: política, valor y política factorizada.
 3. **Seleccionar varias semillas fijas (10).**
 4. **Para cada semilla, repetir los siguientes pasos para eliminar el factor de aleatoriedad:**
 - Para cada versión de las mencionadas anteriormente:
 - a) Ejecutar un entrenamiento completo, manteniendo todos los hiperparámetros fijos.
 - b) Registrar y guardar resultados: loss de train y validation para cada cabeza, accuracy en train y validation, checkpoint del mejor modelo, numero de parámetros, FLOPs estimados...
 - c) Luego, ejecutar este checkpoint contra el conjunto de test para obtener los resultados finales para estudiar el efecto de cada versión sobre el modelo
 5. **Una vez obtenidos los resultados de test para las semillas, utilizar una serie de tests estadísticos para confirmar / desmentir si la hipótesis es estadísticamente significativa.** Se debe comprobar que la hipótesis se cumple teniendo en cuenta la variabilidad de los resultados entre semillas. Para ello se debe decidir entre tests paramétricos o no paramétricos, demostrando la distribución de los datos, y a continuación realizar una prueba estadística para confirmar / desmentir la hipótesis. **Outputs esperados:** tabla con resultados medios y desviación estándar de cada versión, gráficas que muestren la evolución media y varianza de loss y accuracy (media = línea, varianza = sombreado), y resultados de los tests estadísticos.
 6. **Para testear más allá de la accuracy o loss, hay que enfrentar los modelos entre ellos:** la capacidad de predicción no tiene por qué correlacionarse con la capacidad de juego. Por ello, se llevará a cabo lo siguiente:
 - a) Elegir una sola semilla. Usar esta para todos los modelos.
 - b) A las 4 variantes anteriormente mencionadas, añadir 3 modelos más que servirán de comparativa: un MLP base, un agente heurístico, y un Agente aleatorio.
 - c) Para cada par de modelos, ejecutar un torneo de 100 partidas (50 por cada color).
 - d) Registrar resultados y calcular winrate, ELO estimado, entropía de la política, etc.

- e)* Analizar los resultados. Extraer conclusiones sobre la capacidad de juego entre modelos, y con respecto a los resultados de test obtenidos en el paso anterior.
- f)* Output buscado: una matriz de tamaño $N \times N$ (N = número de modelos) con los resultados de winrate entre cada par de modelos, y una tabla con el ELO estimado de cada modelo.

6. Planificación y seguimiento

La siguiente planificación es una aproximación orientada a la experimentación rápida. Para minimizar la fricción en el desarrollo de los experimentos y la lógica interna del ajedrez Gardner esta se delega a `pyffish`, y del mismo modo, en la medida de lo posible los demás componentes se delegan también. Esta decisión acelera las fases iniciales del proyecto y permite además obtener datos de buena calidad para el preentrenamiento de los modelos. A continuación se presenta el plan de trabajo, dividido en fases:

Fase 0: Investigación previa y documentación inicial

- Investigación del estado del arte en RL y Transformers aplicados al ajedrez.
- Consolidación de objetivos generales y específicos. Creación del marco teórico y práctico para el desarrollo del TFG.

Fase 1: Preparación del Entorno y Generación de Datos Sintéticos

- Integración de `pyffish` para la configuración de la variante `gardner` y validación de movimientos.
- Desarrollo y ejecución de *scripts* de *self-play* basados en evaluaciones a profundidad controlada de Fairy-Stockfish para construir el *dataset* de supervisado inicial.

Fase 2: Arquitectura y Preentrenamiento (Supervisado)

- Implementación del modelo Transformer Encoder y las distintas capas de adaptación (I/O).
- Ejecución de experimentos de aprendizaje supervisado utilizando los *datasets* generados, evaluando la velocidad de convergencia y capacidad de las distintas representaciones de tablero y acción.

Fase 3: Aprendizaje por Refuerzo (Pipeline de Self-Play)

- Construcción del entorno vectorizado para muestreo paralelo guiado por las predicciones de política y valor del Transformer.
- Implementación o integración del algoritmo Actor-Crítico Proximal Policy Optimization (PPO).

Fase 4: Evaluación y Generación de Métricas

- Ejecución de torneos automatizados entre agentes de control (Stockfish limitado, modelos puramente *tabula rasa* frente a modelos preentrenados).
- Extracción y visualización de las métricas de rendimiento para la validación de las hipótesis planteadas.

Fase 5: Documentación y Conclusiones

- Consolidación de los resultados en el documento de Trabajo de Fin de Grado, análisis crítico de las gráficas resultantes y preparación de la defensa.

7. Principales aportaciones

TODO aquí las gráficas!!!

- No introducir conceptos nuevos. directo al grano.
- Agrupar los resultados por cada una de las 3 hipótesis. Insertar las 1 o 2 gráficas correspondientes a cada bloque.

7.1. Resultados generales en el entorno de Minichess

qué gráficas / tablas puedo aportar para esta hipótesis? no es exactamente un problema cuantificable. quizás sirve con algo como: *"Como hemos visto en el desarrollo anterior, el problema del Minichess mantiene suficiente complejidad para ser utilizado como entorno de estudio a pesar de su tamaño reducido y su condición de weakly-solved"*

7.2. Efectos de la representación sobre la eficiencia y calidad del aprendizaje

Para contrastar la **Hipótesis 2**, que plantea que las representaciones estructuradas del tablero y del espacio de acciones mejoran la eficiencia del aprendizaje en comparación con las representaciones planas, se ha llevado a cabo una evaluación sistemática y un análisis estadístico riguroso sobre las 4 variantes de arquitectura bajo 10 semillas aleatorias independientes.

7.2.1. Evaluación en el Conjunto de Test Holdout y métricas utilizadas

Los modelos se evaluaron sobre un conjunto de test holdout que contiene 299,325 posiciones de Gardner Minichess extraídas de forma independiente de las trayectorias de entrenamiento. Se reportan métricas tanto para la cabeza de política (predicción de movimientos) como para la cabeza de valor (predicción de resultados).

Para la política, se miden:

- **Cabeza de Política (Predicción de Movimientos):**
 - **Precision@1:** porcentaje de acierto del modelo al seleccionar la jugada con mayor probabilidad en comparación con la jugada óptima.
 - **Precision@3 y Precision@5:** porcentaje en la que la jugada del conjunto de test se encuentra contenida dentro del conjunto de las 3 o 5 predicciones con mayor probabilidad del modelo.
- **Cabeza de Valor (Predicción del Resultado de la Partida):**
 - **Result Accuracy:** Exactitud en la clasificación del resultado esperado de la posición tras redondear la salida continua a la clase discreta más cercana en $\{-1, 0, 1\}$ (derrota, tablas, victoria).
 - **Value MSE:** Error cuadrático medio de la predicción de la cabeza de valor con respecto al resultado final real de la partida.

La Tabla 1 presenta los resultados de test consolidados promediando el rendimiento final de las 10 semillas independientes para cada variante:

Modelo / Representación	Move Acc (P@1)	Precision@3	Precision@5
<code>simple_nofact</code> (Plano, Simple)	48,63 % $\pm$ 0,17 %	83,93 % $\pm$ 0,12 %	95,17 % $\pm$ 0,05 %
<code>simple_fact</code> (Plano, Factorizado)	47,67 % $\pm$ 0,20 %	83,19 % $\pm$ 0,16 %	94,86 % $\pm$ 0,06 %
<code>spatial_nofact</code> (2D, Simple)	48,49 % $\pm$ 0,22 %	83,81 % $\pm$ 0,19 %	95,11 % $\pm$ 0,10 %
<code>spatial_fact</code> (2D, Factorizado)	48,07 % $\pm$ 0,33 %	83,52 % $\pm$ 0,25 %	95,00 % $\pm$ 0,11 %

(a) Métricas de la cabeza de política (movimientos).

Modelo / Representación	Result Acc	Value MSE
<code>simple_nofact</code> (Plano, Simple)	71,88 % $\pm$ 0,13 %	0,2806 $\pm$ 0,0008
<code>simple_fact</code> (Plano, Factorizado)	71,68 % $\pm$ 0,18 %	0,2815 $\pm$ 0,0013
<code>spatial_nofact</code> (2D, Simple)	80,39 % $\pm$ 0,06 %	0,1546 $\pm$ 0,0008
<code>spatial_fact</code> (2D, Factorizado)	79,83 % $\pm$ 0,24 %	0,1588 $\pm$ 0,0019

(b) Métricas de la cabeza de valor (resultado).

Cadro 1: Rendimiento medio y desviación estándar de las métricas sobre el conjunto de test (10 semillas $\times$ 4 variantes).

7.2.2. Test de Significatividad Estadística

Con el propósito de validar de forma matemáticamente rigurosa que las diferencias observadas entre las arquitecturas de ablación no se deben a fluctuaciones aleatorias provocadas por la semilla de inicialización, se aplicó un análisis estadístico compuesto por un test de Shapiro-Wilk para comprobar la normalidad, un test ANOVA de una vía para evaluar la significatividad de las diferencias globales, y comparaciones por parejas post-hoc utilizando el test de Tukey HSD. El análisis completo, junto con las tablas de resultados numéricos de las pruebas, se expone en detalle en el Anexo A.3. A continuación se resumen las conclusiones principales derivadas de estos tests:

- **Predicción de Movimiento (Política):** El test de Shapiro-Wilk no permite rechazar la hipótesis de normalidad ($p = 0,1272 > 0,05$), lo que valida la aplicación del test paramétrico ANOVA. El ANOVA confirma la existencia de diferencias altamente significativas entre modelos ($p = 6,92 \times 10^{-10}$), por lo que procede realizar un análisis comparativo por parejas. Las comparaciones post-hoc de Tukey HSD muestran que:
 - No existen diferencias estadísticamente significativas en la exactitud del movimiento entre la representación plana y la espacial (comparación entre `simple_nofact` y `spatial_nofact`, con diferencia de $-0,14\%$ y $p_{adj} = 0,5926$).
 - La factorización del espacio de acciones degrada significativamente el rendimiento de la política, reduciendo la precisión tanto en el modelo plano ($-0,96\%$, $p_{adj} = 0,0$) como en el espacial ($-0,42\%$, $p_{adj} = 0,0036$).
- **Predicción de Resultado (Valor):** El test de Shapiro-Wilk detecta una desviación del comportamiento estrictamente normal ($p < 0,05$), no obstante, el test ANOVA global sigue siendo altamente significativo ($p = 2,11 \times 10^{-50}$). El test post-hoc de Tukey confirma que:
 - La representación espacial (2D) aporta una mejora muy significativa a la cabeza de valor. Se obtiene un incremento absoluto de la exactitud de resultado de $+8,51\%$ en el modelo simple ($p_{adj} = 0,0$) y de $+8,15\%$ en el factorizado ($p_{adj} = 0,0$), reduciendo el error cuadrático medio (MSE) casi a la mitad (de 0,2806 a 0,1546).

- Nuevamente, la factorización empeora significativamente el desempeño, registrando la variante sin factorizar `spatial_nofact` un rendimiento superior de +0,56 % con respecto a `spatial_fact` ($p_{adj} = 0,0$).

Los boxplots de la Figura 5 ilustran visualmente la dispersión y consistencia de las métricas en test para las 10 semillas independientes, respaldando las conclusiones de significatividad estadística obtenidas.

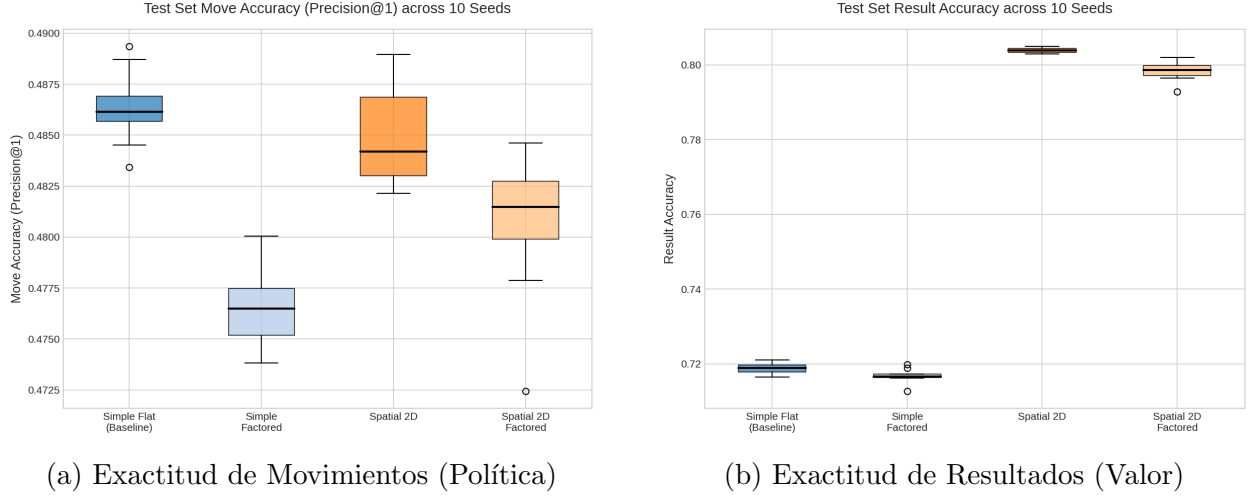


Figura 5: Distribución y variabilidad de las métricas en test sobre las 10 semillas independientes.

7.2.3. Evolución y Curvas de Aprendizaje

Para analizar el comportamiento dinámico durante el entrenamiento de 30 épocas, se promediaron las métricas de validación y entrenamiento a través de las 10 semillas. La Figura 6 muestra las curvas de evolución media (línea continua) y su desviación estándar (área sombreada). Las paradas tempranas aplicadas durante el entrenamiento se gestionan manteniendo el último estado registrado hasta el final del estudio (carry forward).

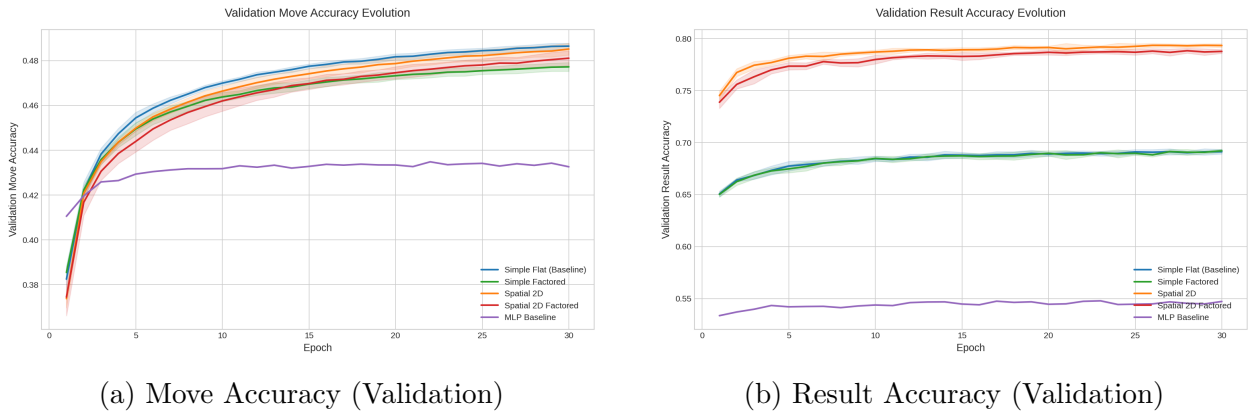


Figura 6: Curvas de aprendizaje promediadas con sombreado de desviación estándar.

Observamos que la exactitud del movimiento las curvas de los modelos planos y espaciales sin factorizar son prácticamente paralelas e idénticas en convergencia, mientras que la exactitud de resultados muestra que el modelo espacial converge a una asíntota notablemente superior a la del modelo plano.

7.2.4. Enfrentamiento en Juego Real: Configuración y Resultados del Torneo

Aunque la evaluación anterior sobre el conjunto de test permite medir la capacidad predictiva de los modelos en situaciones aisladas, el objetivo final del agente es interactuar con el jugador opuesto y ganar partidas reales de ajedrez, y las métricas no tienen por qué correlacionarse directamente con la capacidad de juego del agente. Con este fin, se diseñó un torneo round-robin enfrentando directamente a todos los agentes entre sí, para mostrar la capacidad de juego real.

Configuración Experimental del Torneo: El torneo se ejecutó fijando una serie de parámetros para garantizar la robustez de las comparaciones. Se disputaron 100 partidas por enfrentamiento, divididas equitativamente entre 50 partidas con blancas y 50 con negras para mitigar sesgos de color, con una única semilla (semilla 1) para todas las partidas. Se fijó un límite de 100 movimientos por partida (50 jugadas completas, en caso de alcanzar el límite se considera tablas) y se estableció una temperatura $\tau = 0,1$ para el muestreo de movimientos, y para evitar una política completamente codiciosa.

Es importante explicar el mecanismo de selección de jugadas de los agentes Transformer. En cada turno, el agente utiliza su cabeza de política para extraer las k mejores jugadas legales con mayor probabilidad, donde $k = \min(6, \text{número de jugadas legales})$. A continuación, genera los FEN correspondientes a los tableros resultantes de dichas jugadas y los introduce en lote (batch) a través de la cabeza de valor. El agente selecciona finalmente el movimiento que minimiza la predicción de valor del rival (equivalente a maximizar su propio valor debido al carácter de suma cero del juego). Esto equivale a una búsqueda de profundidad 1 en el árbol de juego, y es similar al proceso de AlphaZero, en el que la red de política se usa para reducir la anchura de la búsqueda MCT, y la red de valor se usa para reducir la profundidad.

El torneo incluye a los 4 modelos del estudio de ablación (entrenados bajo la Semilla 1), una red MLP de referencia (MLP_Baseline, con un tamaño oculto de 1024 neuronas, tasa de aprendizaje de 0.002 y entrenada con un tamaño de lote de 512), un agente de movimientos uniformemente aleatorios (RandomAgent), y un agente heurístico codicioso (HeuristicAgent) que prioriza la captura inmediata de piezas enemigas ponderadas por su valor convencional de material (Dama=9, Torre=5, Alfil/Caballo=3, Peón=1).

Estimación del Rating Elo: Los resultados del torneo se modelan mediante el sistema de Elo, que estima la habilidad relativa de los agentes basándose en los emparejamientos uno a uno. Específicamente, se ajusta un modelo de Bradley-Terry mediante estimación de máxima verosimilitud (MLE). El modelo Bradley-Terry estima la probabilidad esperada de que el agente i gane al agente j ^[19] como:

$$P(i > j) = \frac{e^{R_i}}{e^{R_i} + e^{R_j}} = \frac{1}{1 + 10^{(R_j - R_i)/400}}$$

donde R_i y R_j son los ratings Elo de los agentes. Para calibrar la escala, se ancla el rating del agente RandomAgent a un valor de 1000. Los ratings Elo estimados y la entropía de la política media para cada modelo se presentan en la Tabla 2.

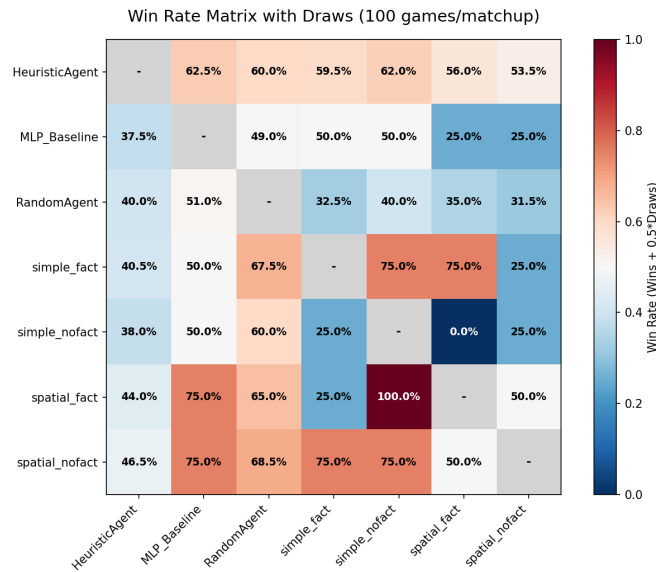
La Figura 7 muestra las matrices de tasa de victoria resultantes, tanto considerando las tablas (wins + 0.5*draws) como ignorando las tablas (decisive games).

7.2.5. Discusión de los Resultados

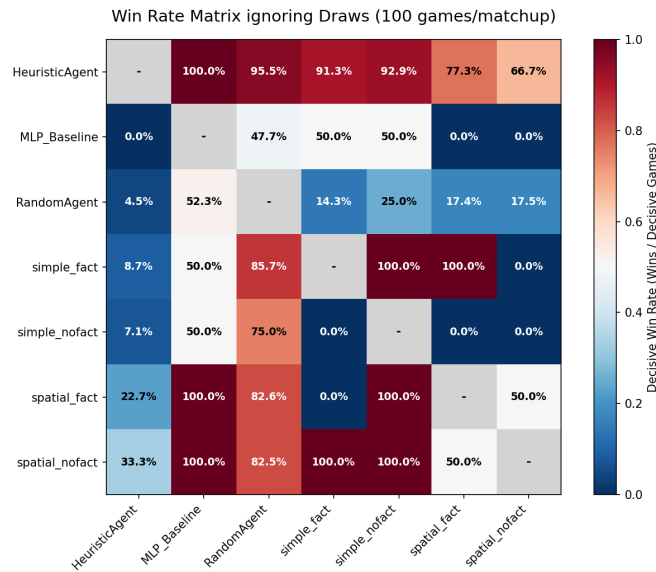
TODO redactar esto mejor. sacar mejores conclusiones

Agente	Arquitectura	Elo Estimado	Entropía Media
spatial_nofact	Transformer (2D, Simple)	1168.8	1.4067
spatial_fact	Transformer (2D, Factored)	1135.2	1.4230
HeuristicAgent	Heurística de Captura	1129.4	N/A
simple_fact	Transformer (Plano, Factored)	1107.8	1.2665
MLP_Baseline	MLP Multicapa Base	1007.0	1.5113
RandomAgent	Aleatorio	1000.0	N/A
simple_nofact	Transformer (Plano, Simple)	964.7	1.3037

Cadro 2: Clasificación final y Elo estimado de los agentes tras el torneo round-robin de 100 partidas.



(a) Win Rate (con tablas)



(b) Win Rate (solo victorias / derrotas)

Figura 7: Matrices de winrate cruzado del torneo de 100 partidas.

Analizando los resultados obtenidos, podemos extraer una serie de conclusiones interesantes. Las más relevantes son la **confirmación de que una mejor representación ayuda al modelo**; el agente `spatial_nofact` obtiene la mejor puntuación en el torneo, con 1168.8 de Elo, superando significativamente a la versión plana `simple_nofact` (964.7 Elo, una diferencia de +204 puntos de Elo). Esta diferencia de rendimiento se alinea con el incremento del +8,51 % observado en la exactitud del valor durante el test supervisado, y con los resultados obtenidos en las tablas de la tabla 1 y la figura 5, y también señala a que la capacidad para estimar el valor de la posición resulta en mejor capacidad de juego.

Esto demuestra que **añadir información espacial ayuda al modelo a obtener una mejor representación interna** y por tanto a tomar mejores decisiones. Esto concuerda con los resultados de estudios previos como Czech et al.^[20] y Monroe et al.^[8], que demuestran, respectivamente, que sigue existiendo cierta importancia en la ingeniería de características (especialmente en dominios específicos como el ajedrez) a pesar del poder de los Transformers, y que los sesgos inductivos (tanto a nivel de input como de codificación posicional) son potentes a la hora de mejorar el rendimiento en dominios específicos. Esto también se alinea con la intuición general de que los Transformers son muy sensibles a la representación de entrada y de salida (codificación de input y función de pérdida), como se menciona recientemente en^[21].

Por otro lado, observamos que **el uso de una cabeza extra de política no mejora el rendimiento del agente e incluso empeora**, tanto para los resultados de valor como de movimiento. Hay varias razones que pueden explicar este fenómeno:

1. Por el diseño de la cabeza de política, ya existe un sesgo inductivo muy similar al de esta cabeza factorizada, que viene dado por el uso de la proyección a un vector de origen y destino y la posterior batch matrix multiplication, lo que puede anular su efecto sobre la representación interna.
2. La función de pérdida utilizada de valor tiene una magnitud aproximadamente 5 veces menor que la de política. Si a esta función de pérdida añadimos la función de pérdida de la política factorizada, aún reducida al multiplicar por 0.5, es posible que esto resulte en un desbalance de la pérdida, provocando que el modelo reciba más "señal" de la política y menos del valor.
3. Finalmente, para explicar el empeoramiento de la política factorizada sobre los movimientos, una posibilidad es que este se deba a la introducción de un sesgo de independencia condicional. Al forzar a la red a predecir mediante tres sub-cabezas independientes (casilla de origen, casilla de destino y tipo de promoción), se introduce un sesgo inductivo de independencia condicional en la probabilidad conjunta de la jugada. Para el aprendizaje supervisado puro, esto puede ser un lastre: a pesar de resultar más informativo, acertar una jugada de profundidad de búsqueda 4 es más difícil si la red tiene que coordinar el acierto de tres variables discretas por separado en lugar de clasificar limpiamente un único vector unificado de 704 logits.

Observamos también en el torneo, que **el agente heurístico de capturas (HeuristicAgent, 1129.4 Elo) obtiene puntuaciones más altas de lo inicialmente esperado**. Esto se debe sobre todo a los datos con los que han sido entrenados los modelos: posiciones con capturas y situaciones tácticas han sido filtradas y hacen que los modelos hayan visto pocos jaques, capturas, etc, y por tanto tengan dificultades para responder ante estas, ya que se encuentran fuera de la distribución de entrenamiento. Además, en un tablero reducido de

5×5 , la proximidad constante de las piezas hace que cualquier despiste táctico de captura sea castigado al instante. A pesar de su simplicidad, el heurístico supera al baseline MLP y al modelo plano simple. Sin embargo, los modelos espaciales `spatial_nofact` (1168.8 Elo) y `spatial_fact` (1135.2 Elo) logran superar al heurístico. Esto demuestra que los agentes basados en Transformer espacial han aprendido conceptos posicionales más sutiles (estructura de peones, control del centro y seguridad del rey) que les permiten maniobrar y tender trampas incluso sin búsqueda.

Uno de los resultados más llamativos del torneo es el rendimiento de `simple_nofact`. En la evaluación supervisada estática de test, este modelo obtiene la mayor precisión de movimientos (Move Accuracy $P@1 = 48.63\%$) y de resultados (Result Accuracy = 71.88%) de toda la categoría plana. No obstante, en juego real su rendimiento colapsa por completo, cayendo a 964.7 de Elo, situándose incluso por debajo del agente aleatorio (`RandomAgent`, 1000 Elo) y de la red MLP (`MLP_Baseline`, 1007 Elo).

Esto ilustra una cierta desconexión entre las métricas estáticas supervisadas y la capacidad de juego interactiva, explicable por factores como la acumulación de errores gradual, que puede llevar a posiciones *Out-of-Distribution* (OOD), o a la alta tasa de empates.

Un **detalle estadístico relevante** surge al analizar los enfrentamientos cruzados (Figura 7). El campeón `spatial_nofact` obtuvo una tasa de victoria del 46,5 % contra `HeuristicAgent` (es decir, el heurístico ganó el enfrentamiento directo por un estrecho margen). Sin embargo, `spatial_nofact` obtiene un Elo superior (1168.8) al de `HeuristicAgent` (1129.4). Esto se debe a la consistencia global: el modelo espacial es dominante contra los Transformers planos y el MLP (logrando por ejemplo un 75 % de victorias contra `simple_fact` y `simple_nofact`), mientras que el heurístico muestra un desempeño más irregular y menos dominante contra el resto de la tabla.

Cabe destacar, finalmente, que el ajedrez Gardner es un juego con tendencia al empate, dado que al ser un tablero tan pequeño situaciones de ahogado son muy comunes y el juego tiende a posiciones donde ambos jugadores se "traban", evitando dar ventajas al otro. Esto se puede ver claramente tanto en el torneo como en las estadísticas del dataset, donde la tasa de empates es significativamente alta.

7.2.6. Conclusión sobre la Hipótesis 2

A partir de los resultados experimentales obtenidos, podemos concluir lo siguiente respecto a la **Hipótesis 2**:

- **Se confirma parcialmente:** La introducción de una representación espacial bidimensional (inductivo bias) mejora de manera estadísticamente significativa el aprendizaje del valor (+8.51 % de exactitud). Sin embargo, esta mejora espacial no se traduce en un incremento significativo en la precisión del movimiento supervisado (resultado de la cabeza de política).
- **Se refuta respecto al espacio de acciones:** El uso de un espacio de acciones factorizado (casillas origen y destino independientes) no solo no mejora el rendimiento, sino que perjudica significativamente tanto la precisión del movimiento como la del valor, introduciendo una complejidad redundante.
- **Resultados de Juego:** El mejor modelo obtenido supera en Elo a un modelo con conocimiento previo del valor de las piezas, aunque la tasa de victorias sea ligeramente

inferior contra el mismo.

7.3. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo

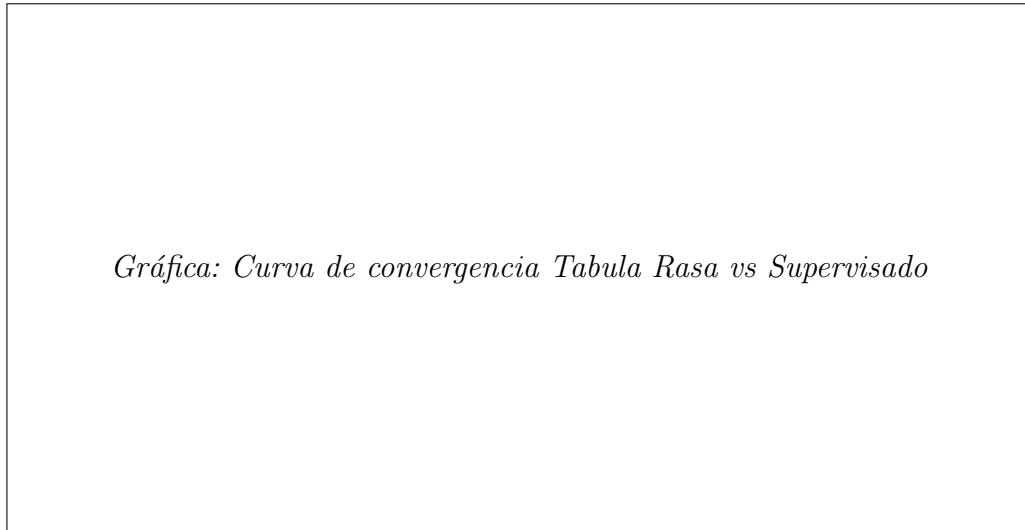


Figura 8: Tasa de victoria y avance del ELO estimado a lo largo del entrenamiento.

Inicialización del Agente	Epochs al pico ELO	ELO final
Tabula rasa		
Pre-entrenado (Supervisado)		

Cadro 3: Comparativa de convergencia pre-entrenamiento vs tabula rasa (Hipótesis 3).

8. Conclusiones

TODO

- Análisis crítico de las gráficas, lectura matemática.
- ¿Se confirman las hipótesis o se desmienten?

9. Vías de trabajo futuro

TODO Lo que me quedó en el tintero

- **TODO: posible RoPE??** (referencia pytorch)
- en vez de factorizar la política, hacerlo con el valor! usar una cabeza auxiliar que sea categórica en lugar de regresiva
- equilibrar el valor de las pérdidas: dar mas peso a value loss
- dado el alcance, quizás habría sido mejor entrenar con un dataset sin filtrar, que incluya capturas y jaques...

- puede que girar el tablero fuera buena idea

TODO limitaciones del enfoque y posibles mejoras

Referencias

- [1] M. R. Segal, “Chess, Chance and Conspiracy,” *Statistical Science*, vol. 22, n.º 1, feb. de 2007, ISSN: 0883-4237. DOI: 10.1214/088342306000000574. URL: <http://dx.doi.org/10.1214/088342306000000574>.
- [2] R. S. Sutton. “The Bitter Lesson.” (2019), URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [3] D. Silver, A. Huang, C. J. Maddison et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, n.º 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961. URL: <https://www.nature.com/articles/nature16961>.
- [4] D. Silver, T. Hubert, J. Schrittwieser et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, n.º 6419, pp. 1140–1144, 2018. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404>.
- [5] J. Jumper, R. Evans, A. Pritzel et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, n.º 7873, pp. 583–589, 2021. URL: <https://www.nature.com/articles/s41586-021-03819-2>.
- [6] Y. Nasu, “Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi,” en *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 1–7. URL: <https://github.com/asdfjkl/nnue>.
- [7] A. Ruoss, G. Delétang, S. Medapati et al., *Amortized Planning with Large-Scale Transformers: A Case Study on Chess*, 2024. arXiv: 2402.04494 [cs.LG]. URL: <https://arxiv.org/abs/2402.04494>.
- [8] D. Monroe e P. A. Chalmers, *Mastering Chess with a Transformer Model*, 2024. arXiv: 2409.12272 [cs.LG]. URL: <https://arxiv.org/abs/2409.12272>.
- [9] D. Monroe, G. Eilender, P. Chalmers, Z. Tang e A. Anderson, *Chessformer: A Unified Architecture for Chess Modeling*, 2026. arXiv: 2605.19091 [cs.LG]. URL: <https://arxiv.org/abs/2605.19091>.
- [10] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons e S. Russell, *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*, 2024. arXiv: 2406.00877 [cs.LG]. URL: <https://arxiv.org/abs/2406.00877>.
- [11] Daniel Monroe. “Transformer Progress in Leela Zero.” (2024), URL: <https://lczero.org/blog/2024/02/transformer-progress/>.
- [12] D. Pritchard e J. Beasley. “Encyclopedia of Chess Variants.” (2007), URL: <https://www.jsbeasley.co.uk/encyc/encyc.pdf>.
- [13] M. Mhalla e F. Prost, “Gardner’s Minichess Variant is solved,” *CoRR*, vol. abs/1307.7118, 2013. arXiv: 1307.7118. URL: <http://arxiv.org/abs/1307.7118>.
- [14] C. E. Shannon, “Programming a Computer for Playing Chess,” *Philosophical Magazine*, vol. 41, n.º 314, pp. 304–317, 1950.
- [15] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention is All you Need,” en *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio et al., eds., vol. 30, Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, “Proximal Policy Optimization Algorithms,” *CoRR*, vol. abs/1707.06347, 2017. arXiv: 1707.06347. URL: <http://arxiv.org/abs/1707.06347>.

- [17] R. Xiong, Y. Yang, D. He et al., “On Layer Normalization in the Transformer Architecture,” *CoRR*, vol. abs/2002.04745, 2020. arXiv: 2002.04745. URL: <https://arxiv.org/abs/2002.04745>.
- [18] A. Gupta, A. Ozdemir e G. Anumanchipalli, *Geometric Interpretation of Layer Normalization and a Comparative Analysis with RMSNorm*, 2025. arXiv: 2409.12951 [cs.LG]. URL: <https://arxiv.org/abs/2409.12951>.
- [19] Wikipedia contributors. “Bradley-Terry model - Wikipedia.” Consultado o 1 de xullo de 2026. (2026), URL: https://en.wikipedia.org/wiki/Bradley%E2%80%93Terry_model#Definition.
- [20] J. Czech, J. Blüml, K. Kersting e H. Steingrimsson, *Representation Matters for Mastering Chess: Improved Feature Representation in AlphaZero Outperforms Switching to Transformers*, 2024. arXiv: 2304.14918 [cs.AI]. URL: <https://arxiv.org/abs/2304.14918>.
- [21] B. Peng, T. Gigant e J. Quesnelle, *Efficient Pre-Training with Token Superposition*, 2026. arXiv: 2605.06546 [cs.CL]. URL: <https://arxiv.org/abs/2605.06546>.
- [22] V. Godbole, G. E. Dahl, J. Gilmer, Z. Nado e C. J. Shallue. “Deep Learning Tuning Playbook.” Google published this outside github too: <https://developers.google.com/machine-learning/guides/deep-learning-tuning-playbook>. (2023), URL: https://github.com/google-research/tuning_playbook.

A. Anexos

Cosas a incluir:

- instrucciones reproducibilidad: configuración del entorno, los comandos de consola para lanzar los scripts de entrenamiento y generación de datos...
- pseudocódigos completos
- tablas de hiperparámetros wandb
- partes importantes de código
- explicaciones más en profundidad de cosas que puedan preguntar en la defensa

A.1. Hiperparámetros del Entrenamiento y Estudio de Ablación

En esta sección se detallan los hiperparámetros fijos empleados en las fases de entrenamiento y, en particular, en el estudio de ablación de arquitectura diseñado para responder a la Hipótesis 2. Este estudio sistemático varía la representación del tablero (*simple* vs. *spatial*) y la utilización o no de la cabeza de política factorizada, repitiéndose bajo un conjunto de semillas aleatorias distintas para evaluar la estabilidad del aprendizaje y mitigar efectos del ruido estocástico.

Siguiendo las recomendaciones del *Deep Learning Tuning Playbook* de Google^[22], para realizar comparaciones rigurosas sobre las variables científicas del modelo se mantuvieron fijos los hiperparámetros. El uso de un tamaño de lote grande ($B = 16384$) maximiza el paralelismo y la estabilidad del gradiente en la GPU pero requiere un ajuste de los hiperparámetros del optimizador (tasa de aprendizaje y estabilidad numérica) obtenidos mediante un proceso de ajuste de Optuna.

Cabe destacar el tamaño de la dimensión de embedding en 64. Este es un valor relativamente bajo (modelos como GPT-2 tienen una dimensión de embedding de 768), pero es adecuado dadas las limitaciones computacionales. Testing informal demostró que aumentar la dimensión de embedding aporta mejoras significativas, junto con el aumento del número de datos de entrenamiento (siguiendo la intuición de las leyes de escalado). El aumento del número de bloques transformer también mejora el rendimiento, pero todo esto a costa de un mayor tiempo de entrenamiento.

La Tabla 4 detalla la configuración y valores específicos de estos hiperparámetros fijos empleados durante las 30 épocas del estudio de ablación.

Los hiperparámetros fijos se eligieron también siguiendo las recomendaciones del *tuning playbook* de DeepMind: modelos simples al inicio, learning rate constante para evitar complicaciones, maximizar el batch size para un entrenamiento más rápido, tunear solo los hiperparámetros importantes para el batch (parámetros de optimizador, learning rate). "*Our guiding principle is to always use simplest, lowest resource configuration that yields good results.*"

El cambio de batch size requirió de ajustes varios, por lo mencionado en la guía: "*The hyperparameters that interact most strongly with the batch size, and therefore are most important to tune separately for each batch size, are the optimizer hyperparameters (e.g. learning rate, momentum) and the regularization hyperparameters.*"

Hiperparámetro (Símbolo)	Valor	Descripción / Contexto Experimental
Dataset de entrada	d4_val.txt	Dataset filtrado obtenido a partir de partidas generadas a profundidad 4.
Split train/validation	97 % / 3 %	División del dataset en conjuntos de entrenamiento y validación.
Dimensión de embedding (d_k)	64	Dimensión de los embeddings de entrada y proyección interna.
Número de bloques (N)	3	Número de bloques encoder del Transformer en el backbone.
Épocas de entrenamiento	30	Pasadas completas sobre el dataset.
Tamaño de lote (B)	16384	Batch size grande para aprovechamiento de GPU.
Backend de atención	math	Kernels de SDPA optimizados para Tensor Cores.
Autocast de precisión	none	Sin autocast, entrenamiento en float32.
Precisión de matmul	high	Precisión para operaciones de multiplicación matricial con Tensor Cores.
Tasa de aprendizaje (lr)	0.002842	Tasa de aprendizaje optimizada tras la exploración de hiperparámetros con Optuna.
Weight decay (λ)	2×10^{-5}	Decaimiento de pesos (penalización L2) para AdamW.
AdamW β_1	0.9244	Coefficiente de promedio móvil del gradiente de primer orden en AdamW.
AdamW β_2	0.999	Coefficiente de promedio móvil del gradiente de segundo orden (por defecto).
AdamW ϵ	$3,6618 \times 10^{-9}$	Parámetro de estabilidad numérica para evitar divisiones por cero.
Semillas aleatorias ($Seed$)	42, 123, 2026, 777, 8765	Múltiples semillas para evaluar estabilidad estocástica de los resultados.

Cadro 4: Hiperparámetros fijos empleados en las ejecuciones del estudio de ablación de arquitectura.

Cabe destacar que los resultados obtenidos se obtuvieron para un conjunto fijo de hiperparámetros. Esto no quita que puedan existir otros conjuntos de hiperparámetros que den mejores resultados, pero para el estudio de ablación realizado no se variaron los hiperparámetros más allá de los que se querían estudiar. No se asegura independencia condicional entre distintos conjuntos de hiperparámetros, y esto limita la extrapolación de resultados a otros conjuntos de hiperparámetros. *"Fixed hyperparameters will have their values fixed in the current round of experiments. These are hyperparameters whose values do not need to (or we do not want them to) change when comparing different values of the scientific hyperparameters. By fixing certain hyperparameters for a set of experiments, we must accept that conclusions derived from the experiments might not be valid for other settings of the fixed hyperparameters. In other words, fixed hyperparameters create caveats for any conclusions we draw from the experiments."* Referencia: Deep Learning Tuning Playbook, Google 2023

A.2. Formulación Matemática de las Funciones de Pérdida

Las pérdidas individuales que componen la función de optimización global durante la fase de aprendizaje supervisado de los modelos se formulan matemáticamente de la siguiente manera:

1. **Pérdida de la Cabeza de Política Principal ($\mathcal{L}_p$):** Se calcula mediante entropía cruzada multiclase clásica (*Cross-Entropy Loss*) aplicada sobre el vector de 704 logits de movimientos legales. Dado el movimiento objetivo $y \in \{1, \dots, 704\}$ (representado como un vector *one-hot* $\mathbf{y}$) y los logits predichos por el modelo $\hat{\mathbf{z}} \in \mathbb{R}^{704}$:

$$\mathcal{L}_p = - \sum_{c=1}^{704} y_c \ln \left(\frac{e^{\hat{z}_c}}{\sum_{k=1}^{704} e^{\hat{z}_k}} \right) \quad (3)$$

2. **Pérdida de la Cabeza de Valor ($\mathcal{L}_v$):** Se evalúa mediante el error cuadrático medio (*Mean Squared Error*, MSE) entre la predicción continua de valor $\hat{v} \in [-1, 1]$ (obtenida tras proyectar linealmente el token **CLS** y aplicar una activación tanh) y el resultado objetivo de la partida $v \in \{-1, 0, 1\}$ (donde -1 indica derrota de las blancas, 0 indica tablas y 1 indica victoria de las blancas):

$$\mathcal{L}_v = (v - \hat{v})^2 \quad (4)$$

3. **Pérdida Auxiliar de la Política Factorizada ($\mathcal{L}_{\text{aux}}$):** Para los modelos factorizados, esta pérdida se desglosa en la suma de tres entropías cruzadas independientes aplicadas sobre las predicciones de la casilla de origen ($\mathcal{L}_{\text{from}}$), casilla de destino ($\mathcal{L}_{\text{to}}$) y el tipo de pieza de promoción ($\mathcal{L}_{\text{promo}}$):

$$\mathcal{L}_{\text{aux}} = \mathcal{L}_{\text{from}} + \mathcal{L}_{\text{to}} + \mathcal{L}_{\text{promo}} \quad (5)$$

donde cada término individual se formula empleando entropía cruzada convencional.

A.2.1. Evolución de las Pérdidas por Cabeza durante el Entrenamiento

Para analizar cómo cada cabeza de salida aprende las características supervisadas en función de su arquitectura, la Figura 9 muestra la evolución de cada componente de pérdida en el conjunto de entrenamiento.

Se puede observar claramente que:

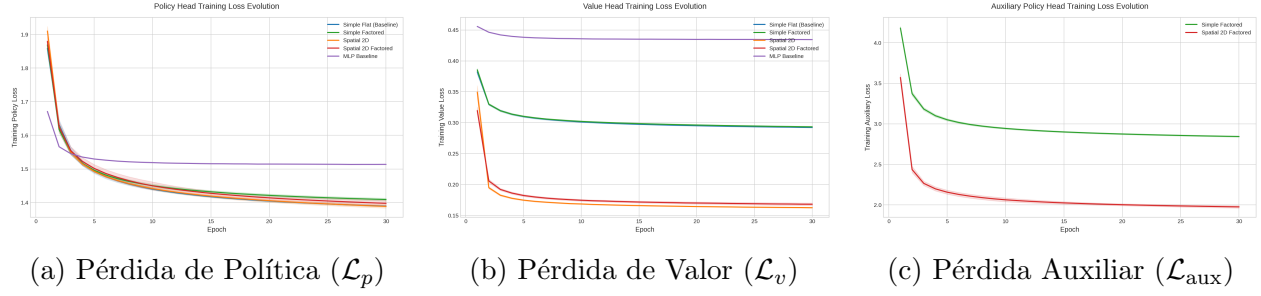


Figura 9: Evolución de las pérdidas de entrenamiento desglosadas por cabeza de salida en el estudio de ablación (media de 10 semillas $\pm$ desviación estándar).

- La pérdida de la cabeza de política ($\mathcal{L}_p$) converge a un valor similar para todos los modelos basados en Transformer, siendo notablemente menor que el baseline MLP. El uso de la cabeza auxiliar factorizada (`simple_fact` y `spatial_fact`) no altera sustancialmente la convergencia de la política principal.
- La pérdida de la cabeza de valor ($\mathcal{L}_v$) muestra la superioridad de las codificaciones de entrada espaciales (`spatial_nofact` y `spatial_fact`), las cuales logran minimizar la pérdida a valores cercanos a 0.28, frente al 0.31 de los modelos planos.
- La pérdida auxiliar ($\mathcal{L}_{aux}$) disminuye progresivamente durante el entrenamiento para las dos arquitecturas factorizadas, indicando que el modelo aprende a correlacionar y estructurar el origen-destino de los movimientos.

A.3. Resultados Detallados de los Tests de Significatividad Estadística

En esta sección se reportan de manera detallada las tablas y p-valores de los análisis estadísticos realizados sobre el estudio de ablación (Hipótesis 2) para las 10 semillas independientes.

A.3.1. Test de Normalidad (Shapiro-Wilk)

Se evalúa la hipótesis nula de que los datos proceden de una distribución normal. Para la exactitud de movimientos (política), se obtiene; p -valor = 0,127205. Al ser $p > 0,05$, no se rechaza la hipótesis nula de normalidad, lo que fundamenta el uso de la prueba ANOVA paramétrica, que requiere que los datos sigan una distribución normal.

Para la exactitud del resultado (valor), el p-valor es prácticamente nulo ($p < 10^{-5}$). Esto indica una desviación de la normalidad teórica, por lo que no se puede usar ANOVA para comparar las medias de los grupos de valor. Aun así, dado el tamaño de la muestra y la magnitud de la diferencia estadística en los resultados que se observa en la figura 5b, y el hecho de que ANOVA es bastante robusto a las violaciones de normalidad bajo ciertas condiciones, y aún sin alcanzar las 30 muestras recomendadas para que el teorema central del límite sea efectivo, el test estadístico puede considerarse adecuado. Por simplicidad se mantiene ANOVA y Turkey para ambos casos, pero pruebas utilizando tests no paramétricos (Kruskal-Wallis y Mann-Whitney U) para el resultado de la cabeza de valor confirman la misma hipótesis obtenidas. **habrá que ver si esto es riguroso...**

A.3.2. Test de Diferencias Globales (One-Way ANOVA)

El valor p devuelto por el test de ANOVA informa sobre la probabilidad de observar los datos observados, o unos más extremos, si la hipótesis nula (que todas las medias son iguales) fuera cierta. Por lo tanto, p -valores pequeños sugieren que es poco probable que las medias de los grupos sean iguales. Se evalúa si existen diferencias significativas entre las medias de los cuatro grupos comparados:

- **Exactitud de Movimiento (Política):** p -valor = $6,92036 \times 10^{-10}$. Existe una diferencia altamente significativa entre los grupos de política.
- **Exactitud de Resultado (Valor):** p -valor = $2,11741 \times 10^{-50}$. Diferencia enormemente significativa entre las representaciones de valor.

A.3.3. Comparaciones Múltiples por Parejas (Tukey HSD)

Se aplica el test de Tukey HSD con una tasa de error global controlada ($\text{FWER} = 0,05$). Las tablas de diferencias de medias (*meandiff*), p -valores ajustados (*p-adj*), intervalos de confianza y la decisión de rechazo se muestran en las Tablas 5 y 6. Recordemos que *meandiff* es la diferencia entre la media del primer grupo y la del segundo (grupo 2 - grupo 1), por lo que un valor positivo indica que el segundo grupo es mejor que el primero y viceversa. Por otro lado, H_0 sostiene que las medias de los grupos son iguales; si se rechaza quiere decir que existe una diferencia estadísticamente significativa entre las medias de los grupos, cuya "dirección" está indicada por el signo de *meandiff*. El intervalo de confianza informa sobre el rango de valores dentro del cual se encuentra la verdadera diferencia entre las medias de los grupos con un 95 % de confianza. Si el intervalo de confianza incluye el cero, no se puede rechazar la hipótesis nula. Finalmente, $p - adj$ es el p -valor ajustado; indica la probabilidad de rechazar la hipótesis nula.

Grupo 1	Grupo 2	Dif. Medias	p-adj	Int. Conf. 95 %	Rechazo H_0
simple_fact	simple_nofact	+0,0096	0,0000	[+0,0066, +0,0127]	Sí
simple_fact	spatial_fact	+0,0040	0,0055	[+0,0010, +0,0071]	Sí
simple_fact	spatial_nofact	+0,0082	0,0000	[+0,0052, +0,0112]	Sí
simple_nofact	spatial_fact	-0,0056	0,0001	[-0,0086, -0,0026]	Sí
simple_nofact	spatial_nofact	-0,0014	0,5926	[-0,0045, +0,0016]	No
spatial_fact	spatial_nofact	+0,0042	0,0036	[+0,0012, +0,0072]	Sí

Cadro 5: Comparación por parejas post-hoc de Tukey HSD para la exactitud del movimiento (Política).

Grupo 1	Grupo 2	Dif. Medias	p-adj	Int. Conf. 95 %	Rechazo H_0
simple_fact	simple_nofact	+0,0020	0,0664	[-0,0001, +0,0041]	No
simple_fact	spatial_fact	+0,0815	0,0000	[+0,0794, +0,0836]	Sí
simple_fact	spatial_nofact	+0,0871	0,0000	[+0,0850, +0,0892]	Sí
simple_nofact	spatial_fact	+0,0795	0,0000	[+0,0774, +0,0816]	Sí
simple_nofact	spatial_nofact	+0,0851	0,0000	[+0,0830, +0,0872]	Sí
spatial_fact	spatial_nofact	+0,0056	0,0000	[+0,0035, +0,0077]	Sí

Cadro 6: Comparación por parejas post-hoc de Tukey HSD para la exactitud del resultado (Valor).